

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCM-UTE

KHÓA LUẬN TỐT NGHIỆP

**THIẾT KẾ VÀ THI CÔNG THIẾT BỊ THÔNG MINH
PHÁT HIỆN BẤT THƯỜNG TRÊN XE ĐIỆN QUA PHÂN TÍCH
RUNG VÀ ÂM THANH ĐỘNG CƠ**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: ĐÀO VĨNH KHANG

MSSV: 22139030

NGUYỄN HOÀNG NGÂN

MSSV: 22139045

Hướng dẫn: TS. TRƯƠNG QUANG PHÚC

TP. HỒ CHÍ MINH - 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



KHÓA LUẬN TỐT NGHIỆP
THIẾT KẾ VÀ THI CÔNG THIẾT BỊ THÔNG MINH
PHÁT HIỆN BẤT THƯỜNG TRÊN XE ĐIỆN QUA PHÂN TÍCH
RUNG VÀ ÂM THANH ĐỘNG CƠ
NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **ĐÀO VĨNH KHANG**
MSSV: 22139030
NGUYỄN HOÀNG NGÂN
MSSV: 22139045

Hướng dẫn: **TS. TRƯƠNG QUANG PHÚC**

TP. HỒ CHÍ MINH - 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ
THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ
NGÀNH HỆ THỐNG NHÚNG VÀ IOT (ĐT)

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Đào Vĩnh Khang

MSSV: 22139030

Nguyễn Hoàng Ngân

MSSV: 22139045

1. Tên đề tài: Thiết kế và triển khai thiết bị thông minh phát hiện bất thường trên xe điện qua phân tích rung và âm thanh động cơ.

2. Các số liệu, tài liệu ban đầu:

- Lý thuyết về xe điện, IoT, Edge AI, xử lý tín hiệu rung động và âm thanh.
- Các phương pháp trích xuất đặc trưng tín hiệu như FFT, MFCC, Mel-Spectrogram và các mô hình học sâu.
- Bộ dữ liệu rung động và âm thanh phục vụ huấn luyện.
- Tài liệu kỹ thuật của Raspberry Pi 4, cảm biến ICM-42688-P, microphone INMP441 và màn hình cảm ứng WaveShare 7 inch.
- Phác thảo kiến trúc phần cứng, phần mềm và giao diện giám sát của hệ thống.

3. Nội dung thực hiện đề tài:

- Thiết kế và thi công thiết bị IoT tích hợp cảm biến rung động, cảm biến âm thanh và bộ xử lý Raspberry Pi 4.
- Triển khai mô hình học sâu trên nền tảng Edge AI sử dụng TensorFlow Lite.
- Xây dựng giao diện Web Dashboard
- Kiểm thử, đánh giá hiệu năng hệ thống và hoàn thiện báo cáo khóa luận.

4. Sản phẩm:

- Thiết bị thông minh phát hiện bất thường trên xe điện dựa trên cảm biến rung động và âm thanh.
- Phần mềm thu thập dữ liệu, xử lý tín hiệu, nhận dạng trạng thái động cơ và giao diện giám sát.

TRƯỞNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ THI CÔNG THIẾT BỊ THÔNG MINH PHÁT HIỆN BẤT THƯỜNG TRÊN XE ĐIỆN QUA PHÂN TÍCH RUNG VÀ ÂM THANH ĐỘNG CƠ

Sinh viên: + Đào Vĩnh Khang..... MSSV: 22139030

+ Nguyễn Hoàng Ngân..... MSSV: 22139045

Hướng dẫn: TS.Trương Quang Phúc

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, cụ thể phù hợp với xu hướng tiếp cận đối với các ô tô hiện đại.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp phân tích hợp lý, dựa trên cơ sở lý thuyết, có tính mới.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả được trình bày rõ ràng, đầy đủ, có kiểm chứng nhằm đảm bảo tính tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận chặt chẽ, phù hợp cách đặt vấn đề, có đóng góp mới mẻ.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Cấu trúc rõ ràng, phù hợp, đúng định dạng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng giao tiếp, làm việc nhóm hiệu quả, chuyên nghiệp.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Tài liệu tham khảo trích dẫn trung thực, đúng quy định.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Tỉ lệ tương đồng đáp ứng đúng theo quy định của nhà trường.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Đề tài có tính ứng dụng, và phù hợp với xu hướng. Tuy nhiên, thực nghiệm còn nhiều hạn chế, chưa tiếp cận trên các sản phẩm thật.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần và thái độ làm việc nghiêm túc, khả năng nghiên cứu tốt.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: “Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ.

Tp. HCM, ngày 24 tháng 6 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



Trương Quang Phúc

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong quá trình nhóm đề tài hoàn thành đồ án tốt nghiệp, nhóm đã nhận được rất nhiều sự tư vấn, hỗ trợ và lời khuyên từ Thầy Trương Quang Phúc. Nhóm xin chân thành cảm ơn thầy!

Nhóm thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)

Đào Vĩnh Khang

Nguyễn Hoàng Ngân

LỜI CAM ĐOAN

Nhóm thực hiện đồ án tốt nghiệp xin cam đoan rằng khóa luận tốt nghiệp này được thực hiện dựa trên kiến thức và nỗ lực cá nhân, không sao chép nội dung hay kết quả từ bất kỳ đồ án, bài báo khoa học, hay tài liệu nào khác. Các tài liệu tham khảo được sử dụng trong khóa luận đều đã được trích dẫn đầy đủ và chính xác theo quy định.

Nhóm thực hiện đồ án tốt nghiệp
(*Ký và ghi rõ họ tên*)

Đào Vĩnh Khang

Nguyễn Hoàng Ngân

TÓM TẮT NỘI DUNG

Xe điện đang trở thành xu hướng phát triển mạnh mẽ trong ngành giao thông vận tải nhờ các ưu điểm về tiết kiệm năng lượng và giảm phát thải, kéo theo nhu cầu ngày càng cao về giám sát và bảo trì kỹ thuật. Trong quá trình vận hành, động cơ xe điện tiềm ẩn nhiều dạng lỗi cơ học nguy hiểm như hỏng vòng bi, mất cân bằng trục và rung động bất thường, những hư hỏng thường khởi phát từ các vết nứt vi mô khó phát hiện bằng phương pháp kiểm tra truyền thống. Đề tài này trình bày việc nghiên cứu, thiết kế và thi công một hệ thống chẩn đoán lỗi tự động cho động cơ xe điện dựa trên công nghệ cảm biến đa phương thức và trí tuệ nhân tạo triển khai tại biên, nhằm phát hiện sớm các bất thường, hỗ trợ bảo trì dự đoán và nâng cao hiệu quả vận hành.

Hệ thống được xây dựng trên nền tảng phần cứng gồm cảm biến gia tốc ICM-42688-P giao tiếp SPI và microphone kỹ thuật số INMP441 giao tiếp I2S, tích hợp trên bo mạch Raspberry Pi 4 đóng vai trò thiết bị biên xử lý trung tâm. Dữ liệu rung động và âm thanh được thu thập đồng thời, tiền xử lý giúp thu thập các ma trận đặc trưng sau đó được đưa vào mô hình mạng tích chập một chiều song song (Apex Fusion 1D-CNN) để phân loại trạng thái động cơ theo thời gian thực. Nhờ kỹ thuật lượng tử hóa tham số và tối ưu hóa mô hình với TensorFlow Lite, toàn bộ quá trình suy luận được thực thi hoàn toàn tại thiết bị biên với độ trễ luôn bé hơn 100ms và không phụ thuộc vào kết nối đám mây. Kết quả chẩn đoán được trực quan hóa qua giao diện web dashboard hiển thị trên màn hình cảm ứng Waveshare 7 inch gắn trực tiếp trên thiết bị.

Các thử nghiệm thực tế trên mô hình động cơ xe điện cho thấy hệ thống có khả năng nhận diện và phân loại chính xác nhiều trạng thái vận hành khác nhau bao gồm trạng thái bình thường, tải nặng và các dạng lỗi cơ học. Giao diện giám sát hiển thị kết quả phân loại, lịch sử cảnh báo và hỗ trợ cập nhật mô hình từ xa qua cơ chế OTA, đảm bảo hệ thống có thể vận hành độc lập trong điều kiện thực tế mà không cần máy tính ngoài. Đề tài đã chứng minh tính khả thi của việc triển khai Edge AI trên thiết bị nhúng giá thành thấp cho bài toán chẩn đoán lỗi công nghiệp, đồng thời mở ra hướng phát triển tiếp theo trong việc mở rộng tập dữ liệu, nâng cao độ chính xác mô hình và tích hợp hệ thống vào các phương tiện xe điện thực tế.

Mục lục

1	TỔNG QUAN	1
1.1	Giới thiệu	1
1.2	Tình hình nghiên cứu	2
1.2.1	Tình hình nghiên cứu trong nước	2
1.2.2	Tình hình nghiên cứu ngoài nước	3
1.3	Mục tiêu	3
1.4	Phương pháp nghiên cứu	3
1.5	Bố cục nội dung	4
2	Cơ sở lý thuyết	5
2.1	Tổng quan về tình trạng lỗi xe điện	5
2.2	Tổng quan về IoT và Edge AI trong giám sát phương tiện	6
2.3	Cơ sở lý thuyết về rung động cơ học	7
2.4	Cơ sở lý thuyết về âm thanh và tín hiệu âm học	7
2.5	HTML, CSS, JS	8
2.6	Các chuẩn truyền thông	9
2.6.1	Chuẩn truyền thông SPI	9
2.6.2	Chuẩn truyền thông I2S	10
2.6.3	Chuẩn truyền thông HTTP	11
2.6.4	Chuẩn truyền thông HDMI	11
2.6.5	Chuẩn truyền thông USB HID	12
2.7	Thiết bị phần cứng	12
2.7.1	Máy tính Raspberry Pi 4	12

2.7.2	Cảm biến âm thanh INMP441	15
2.7.3	Cảm biến gia tốc ICM42688	17
2.7.4	Màn hình Capacitive Touch Screen LCD 7 inches	19
3	Thiết kế và thi công hệ thống	22
3.1	Yêu cầu thiết kế	22
3.1.1	Yêu cầu về tính năng của sản phẩm	22
3.1.2	Yêu cầu về kỹ thuật	23
3.2	Đặc tả hệ thống	23
3.3	Thiết kế phần cứng	25
3.3.1	Khối nguồn	25
3.3.2	Khối cảm biến	26
3.3.3	Khối xử lý trung tâm	27
3.3.4	Khối giao diện người dùng	27
3.3.5	Máy chủ OTA	28
3.3.6	Sơ đồ nguyên lý	29
3.4	Thiết kế phần mềm	30
3.4.1	Dữ liệu và tiền xử lý tín hiệu	30
3.4.2	Kiến trúc mô hình học sâu	36
3.4.3	Lưu đồ giải thuật chính của khối xử lý trung tâm	42
4	Kết quả	52
4.1	Kết quả hệ thống	52
4.2	Kết quả thực hiện phần cứng	57
4.3	Kết quả nghiên cứu phần mềm	59
4.4	Thảo luận	60
4.4.1	Đánh giá kết quả huấn luyện	60
4.4.2	Đánh giá chi tiết hiệu năng suy luận thực nghiệm	61
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	67

Danh sách bảng

2.1	Thông số kỹ thuật của cảm biến INMP441	16
2.2	Thông số kỹ thuật của cảm biến ICM42688	18
2.3	Thông số kỹ thuật của màn hình Waveshare 7inch HDMI LCD (C) . . .	20
3.1	Kết nối INMP441 với Raspberry Pi 4	29
3.2	Kết nối cảm biến ICM-42688 với Raspberry Pi 4	30
4.1	Tổng hợp độ chính xác trên từng loại mô hình	62
4.2	Tổng hợp so sánh hiệu năng toán học và tài nguyên phần cứng Edge AI .	63

Danh sách hình vẽ

2.1	Sơ đồ kết nối của chuẩn truyền thông SPI	9
2.2	Bảng mạch Raspberry Pi4	13
2.3	Cảm biến âm thanh INMP441	15
2.4	Cảm biến ICM42688	17
2.5	Màn hình Waveshare 7inch HDMI LCD (C)	19
3.1	Sơ đồ khối hệ thống	24
3.2	Sơ đồ kết nối Raspberry Pi và INMP441	29
3.3	Sơ đồ kết nối Raspberry Pi và ICM42688	30
3.4	Lưu đồ thuật toán trích xuất đặc trưng từng nhánh cảm biến	32
3.5	Lưu đồ cơ chế điều hướng dự phòng dữ liệu	35
3.6	Lưu đồ khối trích xuất đặc trưng song song	37
3.7	Lưu đồ khối hợp nhất chú ý chéo thông minh	39
3.8	Lưu đồ tầng phân loại quyết định đầu ra	41
3.9	Lưu đồ giải thuật toàn hệ thống	43
3.10	Lưu đồ giải thuật quá trình thu dữ liệu của cảm biến	45
3.11	Lưu đồ giải thuật quá trình xử lý và nhận dạng tình trạng động cơ	47
3.12	Lưu đồ hàm check_update()	49
3.13	Lưu đồ hàm do_update()	50
3.14	Lưu đồ hàm notify_infer_reload()	51
4.1	Mặt trước sản phẩm	52
4.2	Mặt sau sản phẩm	53

4.3	Trạng thái standby	53
4.4	Hệ thống trả kết quả động cơ ở trạng thái Normal	54
4.5	Động cơ ở trạng thái Heavyload	55
4.6	Nút Install Update sáng lên khi có bản cập nhật mới	56
4.7	Hệ thống ở trạng thái đã cập nhật	56
4.8	Bộ xử lý trung tâm được lắp đặt phía sau	57
4.9	Cảm biến được nối với bộ xử lý trung tâm qua các dây tín hiệu	58
4.10	Các thành phần trên giao diện	59
4.11	Giao diện người dùng	60
4.12	Biểu đồ tiến trình hội tụ	61
4.13	Bảng báo cáo đánh giá hiệu năng suy luận chi tiết trên mô hình đề xuất .	64
4.14	Ma trận nhầm lẫn trên 300 mẫu kiểm thử	65

Danh mục các từ viết tắt

Dưới đây là danh mục các từ viết tắt được sử dụng trong luận văn.

Các từ viết tắt	Định nghĩa
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
CWRU	Case Western Reserve University
DC	Direct Current
FFT	Fast Fourier Transform
FIFO	First In First Out
GPIO	General Purpose Input/Output
HDMI	High-Definition Multimedia Interface
HID	Human Interface Device
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
I3C	Improved Inter-Integrated Circuit
IMU	Inertial Measurement Unit
IoT	Internet of Things
IPS	In-Plane Switching

JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
MEMS	Micro-Electro-Mechanical Systems
MFCC	Mel-Frequency Cepstral Coefficients
OTA	Over-The-Air
PCEN	Per-Channel Energy Normalization
PID	Process Identifier
PoE	Power over Ethernet
PWM	Pulse Width Modulation
RAM	Random Access Memory
ReLU	Rectified Linear Unit
SHA256	Secure Hash Algorithm 256-bit
SIGUSR1	Signal User-defined 1
SNR	Signal-to-Noise Ratio
SPI	Serial Peripheral Interface
STFT	Short-Time Fourier Transform
TCP/IP	Transmission Control Protocol / Internet Protocol
THD	Total Harmonic Distortion
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
URL	Uniform Resource Locator
USB	Universal Serial Bus

Chương 1

TỔNG QUAN

1.1 Giới thiệu

Trong bối cảnh hiện nay, xe điện đang trở thành xu hướng phát triển mạnh mẽ trong ngành giao thông vận tải nhờ các ưu điểm vượt trội như tiết kiệm năng lượng, giảm phát thải khí nhà kính cũng như chi phí vận hành thấp. Tại Việt Nam, số lượng xe điện đang gia tăng nhanh chóng, kéo theo nhu cầu ngày càng cao về giám sát tình trạng vận hành và đảm bảo an toàn kỹ thuật.

Xe điện đang ngày càng được ứng dụng rộng rãi trong đời sống và công nghiệp. Tuy nhiên, trong quá trình vận hành, động cơ xe điện vẫn có thể phát sinh nhiều dạng lỗi như mất cân bằng cơ khí, hỏng vòng bi, lỗi motor, lệch trục, rung động bất thường hoặc các lỗi liên quan đến hệ thống điện. Những lỗi này nếu không được phát hiện sớm sẽ ảnh hưởng đến hiệu suất hoạt động, làm giảm tuổi thọ thiết bị và thậm chí dẫn đến các sự cố nghiêm trọng trong quá trình vận hành.

Bên cạnh đó, các phương pháp kiểm tra và chẩn đoán lỗi truyền thống chủ yếu dựa trên kinh nghiệm của kỹ thuật viên hoặc sử dụng các thiết bị chuyên dụng có giá thành cao. Quá trình này thường đòi hỏi nhiều thời gian, chi phí bảo trì lớn và khó áp dụng đối với các hệ thống quy mô nhỏ hoặc các mô hình nghiên cứu thử nghiệm. Trong khi đó, với sự phát triển nhanh chóng của công nghệ cảm biến, Internet vạn vật (IoT) và trí tuệ nhân tạo, việc xây dựng một hệ thống giám sát và chẩn đoán lỗi tự động có thể được ứng dụng nhằm nâng cao hiệu quả khai thác và bảo trì động cơ xe điện.

Để thực hiện mục tiêu này, nhóm đã tiến hành nghiên cứu và phát triển hệ thống dựa trên sự kết hợp giữa phần cứng thu thập dữ liệu và các thuật toán xử lý. Hệ thống sử dụng cảm biến gia tốc để thu thập tín hiệu rung động từ động cơ xe điện trong quá trình hoạt động. Dữ liệu sau khi thu thập sẽ được tiền xử lý, trích xuất đặc trưng và đưa vào mô hình học máy để phân tích và nhận diện trạng thái hoạt động của động cơ. Thông qua việc phân tích các đặc trưng tín hiệu như miền thời gian, miền tần số hoặc ảnh phổ tín hiệu, hệ thống có thể phát hiện và phân loại chính xác các lỗi khác nhau.

Tóm lại, báo cáo này trình bày việc thiết kế và thi công hệ thống chẩn đoán lỗi động cơ xe điện dựa trên công nghệ cảm biến và trí tuệ nhân tạo. Hệ thống giúp phát hiện sớm các lỗi phát sinh, hỗ trợ bảo trì dự đoán và nâng cao hiệu quả vận hành động cơ. Đồng thời, báo cáo cũng đề cập đến các khó khăn, tiềm năng phát triển và hướng cải tiến trong tương lai nhằm hoàn thiện hệ thống và nâng cao khả năng ứng dụng thực tế.

1.2 Tình hình nghiên cứu

1.2.1 Tình hình nghiên cứu trong nước

Tại Việt Nam, các nghiên cứu liên quan đến giám sát tình trạng phương tiện giao thông chủ yếu tập trung vào các hệ thống chẩn đoán điện tử, giám sát tiêu thụ năng lượng của pin và định vị phương tiện. Một số công trình đã đề cập đến việc sử dụng cảm biến rung động hoặc tín hiệu âm thanh trong chẩn đoán tình trạng máy móc công nghiệp hạng nặng. Tuy nhiên, phạm vi ứng dụng còn hạn chế, đa số xử lý dữ liệu offline và chủ yếu dừng lại ở mức mô phỏng trên máy tính. Đối với xe điện, các nghiên cứu trong nước hiện nay phần lớn tập trung vào thiết kế bộ điều khiển động cơ, hệ thống quản lý pin và tối ưu hiệu suất. Việc ứng dụng phân tích rung và âm thanh kết hợp học máy để chẩn đoán lỗi cơ khí, sau đó tích hợp thành một thiết bị Edge AI độc lập gắn trên xe vẫn còn rất mới mẻ và chưa có giải pháp thương mại hóa rộng rãi.

1.2.2 Tình hình nghiên cứu ngoài nước

Trên thế giới, các phương pháp giám sát tình trạng thiết bị dựa trên phân tích rung và âm thanh đã đạt được nhiều thành tựu lớn. Nhiều trung tâm nghiên cứu uy tín, tiêu biểu như Đại học Case Western Reserve đã xây dựng các bộ dữ liệu chuẩn mực về tín hiệu gia tốc, chứng minh rằng phân tích tần số cộng hưởng có thể chỉ điểm chính xác các vết nứt vi mô trên vòng bi động cơ điện. Đồng thời, xu hướng trong lĩnh vực giám sát tình trạng thiết bị là không sử dụng cảm biến đơn lẻ. Việc dung hợp đặc trưng của hệ số âm thanh học và rung động miền thời gian đã được chứng minh đem lại năng lực kháng nhiễu vượt trội trong các nhà xưởng ồn ào. Bên cạnh đó, việc đưa các mạng nơ-ron sâu xuống chạy trực tiếp trên các vi máy tính như Raspberry Pi đã trở nên khả thi. Các nghiên cứu chỉ ra rằng, nhờ kỹ thuật tối ưu hóa mô hình, thiết bị biên hoàn toàn có thể xử lý luồng dữ liệu cảm biến thời gian thực với độ trễ tính bằng mili-giây, đảm bảo cảnh báo tức thời cho hệ thống.

1.3 Mục tiêu

Mục tiêu của đề tài bao gồm:

- Thiết kế và thi công một thiết bị IoT độc lập có khả năng thu thập tín hiệu rung và âm thanh trong quá trình vận hành của xe điện.
- Nghiên cứu và ứng dụng các kỹ thuật xử lý tín hiệu và trí tuệ nhân tạo để phát hiện bất thường từ dữ liệu rung và âm thanh.
- Triển khai thuật toán phát hiện bất thường trên Raspberry Pi.
- Đánh giá hiệu quả hoạt động của hệ thống.

1.4 Phương pháp nghiên cứu

Để đạt được mục tiêu đề ra, đề tài có sử dụng các phương pháp nghiên cứu lý thuyết và phương pháp thực nghiệm.

1.5 Bố cục nội dung

Báo cáo khóa luận tốt nghiệp của nhóm sẽ bao gồm 5 chương:

- Chương 1: Tổng quan.
- Chương 2: Cơ sở lý thuyết.
- Chương 3: Thiết kế và xây dựng hệ thống.
- Chương 4: Thi công và kết quả.
- Chương 5: Kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết

2.1 Tổng quan về tình trạng lỗi xe điện

Trong kỹ thuật hệ thống hiện đại, việc đảm bảo độ tin cậy và an toàn vận hành càng trở nên cấp thiết trước sự dịch chuyển của ngành công nghiệp ô tô. Khác với phương tiện truyền thống, xe điện sử dụng cấu trúc điện tinh gọn có hệ truyền động đóng vai trò trung tâm chuyển hóa năng lượng. Thành phần cốt lõi chịu tải trọng cơ học lớn nhất trong hệ thống này là cụm ổ bi lăn của động cơ điện và trục dẫn động [1,2]. Về mặt động lực học, vòng bi đảm nhiệm chức năng triệt tiêu ma sát trượt và duy trì chuyển động quay mượt mà.

Chính vì vai trò chịu tải trọng tâm này, hệ truyền động luôn đối mặt với quá trình hao mòn cơ học tất yếu. Dưới tác động vận xoắn và tiếp xúc lăn, cấu trúc vật liệu của vòng bi sẽ chịu sự suy thoái liên tục theo thời gian. Sự suy thoái này tạo ra các vết nứt, xước hoặc rỗ xộp có kích thước siêu nhỏ cả trong và ngoài bề mặt rãnh lăn [1,2]. Nếu không được kiểm soát, chúng sẽ mở rộng quy mô, làm thay đổi đặc tính động học, gây kẹt trục và dẫn đến phá hủy hoàn toàn kết cấu. Để giải quyết bài toán kiểm soát rủi ro cơ học nêu trên, nền tảng kỹ thuật chẩn đoán lỗi cung cấp hai hướng tiếp cận chính. Hướng tiếp cận thứ nhất là dựa vào hệ thống chẩn đoán và giám sát các biến thiên vĩ mô về dòng điện và nhiệt độ. Tuy nhiên, phương pháp này mang tính thụ động và có độ trễ cao và chỉ bộc lộ khi tổn thất cơ học đã ở mức nghiêm trọng. Do đó, hướng tiếp cận thứ hai mang tính ưu việt hơn là giám sát dựa trên trạng thái thông qua phân tích tín hiệu vật lý tần số cao,

khi các chi tiết máy va chạm với vết nứt vi mô, động năng sẽ giải phóng năng lượng lan truyền qua vỏ động cơ dưới dạng gia tốc rung động và bức xạ ra môi trường dưới dạng âm học [3,4]. Việc ứng dụng kết hợp dữ liệu đa phương thức từ cả hai miền tín hiệu rung và âm thanh cung cấp một không gian đặc trưng toàn diện, giúp các thuật toán nhận dạng mẫu chính xác loại lỗi ngay từ giai đoạn khởi phát, đồng thời lọc bỏ nhiễu tạp âm từ môi trường vận hành thực tế.

2.2 Tổng quan về IoT và Edge AI trong giám sát phương tiện

Để hiện thực hóa các phương pháp giám sát trạng thái một cách liên tục và tự động, IoT cung cấp hạ tầng viễn thông và thu thập dữ liệu thiết yếu. Sự tích hợp của các nút mạng IoT được trang bị cảm biến điện tử trực tiếp lên hệ truyền động tạo ra một mạng lưới có khả năng số hóa các biến thiên vật lý theo thời gian thực, đánh dấu bước chuyển dịch quan trọng từ chiến lược bảo trì phục hồi sang mô hình bảo trì dự đoán, giúp tối ưu hóa vòng đời thiết bị. Mặc dù IoT cung cấp hạ tầng kết nối mạnh mẽ, việc áp dụng mô hình kiến trúc xử lý đám mây truyền thống vào bài toán chẩn đoán phương tiện lại gặp phải những giới hạn vật lý nghiêm trọng. Đặc thù của phương pháp giám sát rung động và âm thanh là yêu cầu tần số lấy mẫu tín hiệu rất cao. Việc truyền tải liên tục luồng dữ liệu thô khổng lồ này lên máy chủ đám mây gây ra sự bão hòa băng thông mạng, tiêu hao năng lượng và tạo ra độ trễ phản hồi. Những giới hạn này làm mất đi khả năng cảnh báo tức thời, một yếu tố sống còn đối với an toàn phương tiện giao thông.

Nhằm giải quyết triệt để nút thắt cổ chai về truyền tải dữ liệu, lý thuyết Điện toán biên kết hợp Trí tuệ nhân tạo, gọi tắt là Edge AI, được ứng dụng như một cơ chế xử lý phân tán [5,6]. Trọng tâm của Edge AI là dịch chuyển năng lực tính toán từ trung tâm dữ liệu về sát nguồn phát sinh tín hiệu, cụ thể là các vi xử lý nhúng như Raspberry Pi. Các kiến trúc mạng nơ-ron học sâu sau khi huấn luyện sẽ được định hình lại thông qua kỹ thuật lượng tử hóa tham số giúp nén trọng số toán học của mô hình, giảm thiểu khối lượng tính toán mà không làm suy giảm đáng kể độ chính xác. Kết quả là thiết bị biên có khả năng tự chủ thực thi thuật toán trích xuất đặc trưng đa phương thức và đưa ra quyết định

suy luận với độ trễ chỉ tính bằng mili-giây, đảm bảo hệ thống giám sát vận hành độc lập và chính xác.

2.3 Cơ sở lý thuyết về rung động cơ học

Về mặt động lực học, khi hệ truyền động của xe điện hoạt động, sự tương tác giữa các chi tiết máy luôn tạo ra dao động tuần hoàn ổn định, được gọi là rung động nền. Sự cân bằng này sẽ bị phá vỡ nếu thành phần chịu tải cốt lõi là cụm ổ bi lăn bắt đầu xuất hiện dấu hiệu xuống cấp. Cụ thể, các vết nứt hoặc rỗ xấp siêu nhỏ trên bề mặt rãnh lăn sẽ biến mỗi chu kỳ tiếp xúc của con lăn thành một cú va đập cơ học đột ngột. Chuỗi va chạm lặp đi lặp lại này liên tục kích thích tần số dao động tự nhiên của vỏ động cơ, từ đó sinh ra các xung tín hiệu dốc có biên độ lớn.

Dựa trên các thông số cấu tạo của ổ bi như số lượng con lăn, đường kính con lăn, góc tiếp xúc và tốc độ quay của trục, mỗi dạng hư hỏng sẽ tạo ra một tần số dao động đặc trưng riêng. Các tần số này được sử dụng làm cơ sở để nhận biết vị trí xảy ra lỗi trong ổ bi. Trong thực tế, bốn tần số lỗi quan trọng thường được theo dõi bao gồm lỗi vòng ngoài, lỗi vòng trong, lỗi con lăn và lỗi vòng cách.

Để chuyển đổi các dao động này thành dữ liệu số, cảm biến gia tốc sẽ được lắp đặt trực tiếp trên vỏ máy. Sau khi dữ liệu được thu thập, bước tiếp theo là ứng dụng kỹ thuật Biến đổi Fourier nhanh (Fast Fourier Transform) để dịch chuyển tín hiệu từ miền thời gian sang miền tần số, giúp bóc tách chính xác các đỉnh biên độ vượt ngưỡng [7, 8].

2.4 Cơ sở lý thuyết về âm thanh và tín hiệu âm học

Bên cạnh kênh tín hiệu rung động, một nguồn thông tin khác cũng không kém phần quan trọng là sóng áp suất lan truyền qua không khí, tức là tín hiệu âm học. Điểm vượt trội của phương pháp phân tích âm thanh là khả năng ghi nhận các bức xạ âm tần số cao phát ra từ các vết nứt siêu nhỏ. Đặc biệt là trong giai đoạn đầu của sự cố, khi xung rung động chưa đủ mạnh để kích hoạt cảm biến gia tốc, thì tín hiệu âm học đã cho thấy sự biến đổi rõ rệt.

Mặc dù sở hữu độ nhạy cao, tuy nhiên tín hiệu âm thanh thu bằng microphone thông thường rất dễ bị suy giảm chất lượng do hiện tượng giao thoa và nhiễu tạp âm từ môi trường vận hành xung quanh. Để khắc phục hạn chế này, giải pháp sử dụng dữ liệu đa phương thức được đề xuất, tạo ra không gian đặc trưng toàn diện và tin cậy hơn cho các thuật toán nhận dạng.

Về mặt xử lý toán học, thay vì chỉ sử dụng phổ tần số thông thường, tín hiệu âm học thường được biến đổi sang miền thời gian thông qua hệ số MFCCs hoặc Mel-Spectrogram để mô phỏng lại cách phân tách âm thanh của thính giác, tạo đầu vào tối ưu cho mô hình học sâu [9–11].

2.5 HTML, CSS, JS

Đối với một hệ thống IoT và Edge AI hoàn chỉnh, dữ liệu sau khi suy luận tại biên cần phải được trực quan hóa dữ liệu. Để thực hiện điều này, bộ ba công nghệ Web tiêu chuẩn bao gồm HTML, CSS và JavaScript được lựa chọn để xây dựng giao diện giám sát thời gian thực.

Trước hết, đóng vai trò làm khung sườn cho toàn bộ giao diện là HTML (HyperText Markup Language). Công nghệ này chịu trách nhiệm định cấu trúc cho các vùng hiển thị, từ các bảng nhật ký cảnh báo lỗi, cho đến các phân đoạn hiển thị trạng thái kết nối của thiết bị Edge.

Nhằm tăng tính trực quan cho các thông tin, CSS (Cascading Style Sheets) được tích hợp để thiết lập bố cục và phong cách đồ họa. Không chỉ dừng lại ở mặt thẩm mỹ, CSS còn đóng vai trò phân loại trực quan các trạng thái kỹ thuật của thiết bị thông qua hệ thống mã màu quy ước. Việc thiết lập các dải màu từ an toàn đến nguy hiểm giúp người vận hành nhận diện nhanh chóng mức độ nghiêm trọng của sự cố.

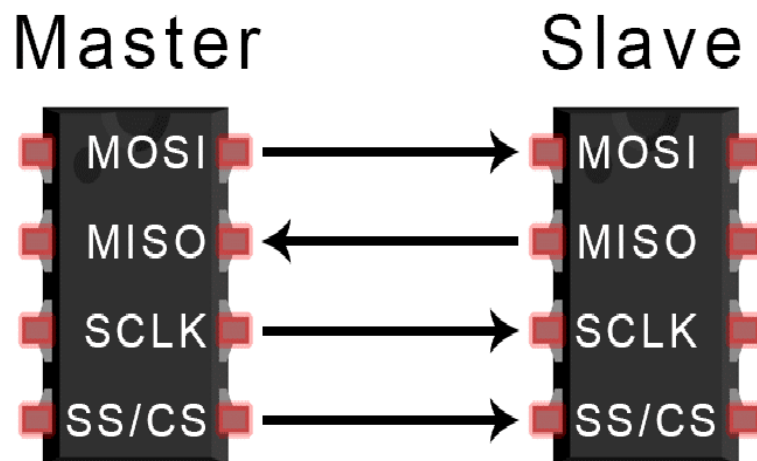
Tuy nhiên, JavaScript mới chính là nhân tố quan trọng nhất mang lại tính tương tác thời gian thực cho hệ thống, cho phép xử lý các tác vụ bất đồng bộ và duy trì các luồng truyền nhận dữ liệu liên tục mà không yêu cầu người dùng phải tải lại trang.

2.6 Các chuẩn truyền thông

2.6.1 Chuẩn truyền thông SPI

Giao thức truyền thông Serial Peripheral Interface (SPI) là một giao thức truyền thông đồng bộ được sử dụng để kết nối và truyền dữ liệu giữa các thiết bị điện tử trong hệ thống nhúng. Nó được phát triển bởi Motorola vào những năm 1980 và sau đó trở thành một tiêu chuẩn ngành công nghiệp.

Về nguyên tắc hoạt động, dựa vào Hình 2.1, bao gồm hai thành phần chính: Master (thiết bị chủ) và Slave (thiết bị con). Master là nguồn điều khiển gửi lệnh và nhận dữ liệu từ Slave. Các Slave là các thiết bị nhận và trả lời các lệnh từ Master. SPI sử dụng hai đường truyền chính: Đường truyền dữ liệu (MOSI - Master Output Slave Input): Master gửi dữ liệu tới Slave thông qua đường này. Đường truyền dữ liệu (MISO - Master Input Slave Output): Slave trả lời Master thông qua đường này. SPI cũng sử dụng các tín hiệu khác như: Đồng hồ (SCK - Serial Clock): Đồng hồ này được Master tạo ra và đồng bộ hóa quá trình truyền dữ liệu giữa Master và Slave. Tín hiệu chọn (SS - Slave Select): Mỗi Slave có một tín hiệu chọn riêng để Master xác định thiết bị nào sẽ giao tiếp.



Hình 2.1: Sơ đồ kết nối của chuẩn truyền thông SPI

Đặc điểm của chuẩn truyền SPI: Tốc độ truyền dữ liệu: SPI cho phép truyền dữ liệu với tốc độ cao. Tuy nhiên, tốc độ truyền dữ liệu phụ thuộc vào tốc độ đồng hồ và cấu hình của hệ thống. Độc lập với giao thức truyền: SPI không yêu cầu giao thức truyền cụ

thể. Nó chỉ cung cấp cơ chế truyền dữ liệu đồng bộ giữa Master và Slave. Linh hoạt trong cấu hình: SPI cho phép cấu hình các tham số như tốc độ truyền, kiểu truyền (full-duplex, half-duplex) và thứ tự truyền (LSB-first, MSB-first). Tương thích ngược: SPI 16 có tính tương thích ngược, cho phép kết nối và giao tiếp giữa các thiết bị SPI từ các nhà sản xuất khác nhau.

2.6.2 Chuẩn truyền thông I2S

Giao thức truyền thông I2S (Inter-IC Sound) là một giao thức truyền thông đồng bộ được sử dụng để truyền dữ liệu âm thanh số giữa các thiết bị điện tử trong hệ thống nhúng. Nó được phát triển bởi Philips vào năm 1986 và được sử dụng rộng rãi trong các ứng dụng âm thanh như DAC, ADC, microphone số và bộ xử lý tín hiệu âm thanh.

Nguyên tắc hoạt động: I2S bao gồm hai thành phần chính: Master (thiết bị chủ) và Slave (thiết bị con). Master là thiết bị tạo tín hiệu clock và điều khiển quá trình truyền dữ liệu. Slave là thiết bị nhận hoặc gửi dữ liệu theo tín hiệu clock từ Master. I2S sử dụng các đường truyền chính như: Đường truyền dữ liệu (SD - Serial Data): dùng để truyền dữ liệu âm thanh số giữa các thiết bị. Đường xung clock bit (BCLK - Bit Clock): do Master tạo ra để đồng bộ từng bit dữ liệu. Đường chọn kênh (LRCLK hoặc WS - Word Select): xác định kênh dữ liệu trái (Left) hoặc phải (Right) trong hệ thống âm thanh stereo. Ngoài ra, một số hệ thống còn sử dụng tín hiệu MCLK (Master Clock) để cung cấp xung clock chính cho các thiết bị âm thanh.

Đặc điểm của chuẩn truyền I2S: Tối ưu cho truyền dữ liệu âm thanh: I2S được thiết kế chuyên biệt cho tín hiệu audio số, hỗ trợ truyền dữ liệu stereo. Đồng bộ cao: Dữ liệu được truyền đồng bộ theo xung clock giúp đảm bảo độ chính xác. Không sử dụng địa chỉ: I2S không có cơ chế địa chỉ hóa như I2C, thường dùng kết nối trực tiếp giữa các thiết bị. Truyền dữ liệu nối tiếp: Giảm số lượng chân kết nối, phù hợp với hệ thống nhúng. Linh hoạt về độ phân giải: Hỗ trợ nhiều chuẩn dữ liệu như 16-bit, 24-bit, 32-bit tùy theo cấu hình hệ thống.

2.6.3 Chuẩn truyền thông HTTP

Giao thức truyền thông HTTP (HyperText Transfer Protocol) là giao thức truyền thông thuộc tầng ứng dụng được sử dụng rộng rãi để trao đổi dữ liệu giữa thiết bị khách (Client) và máy chủ (Server) thông qua mạng Internet hoặc mạng nội bộ. HTTP được phát triển bởi Tim Berners-Lee vào đầu những năm 1990 nhằm phục vụ việc truyền tải tài liệu trên World Wide Web. Ngày nay, HTTP được sử dụng phổ biến trong các hệ thống Web, ứng dụng di động, IoT và các dịch vụ điện toán đám mây.

Nguyên tắc hoạt động: HTTP hoạt động dựa trên mô hình Client – Server. Trong quá trình truyền thông, Client gửi một yêu cầu (Request) đến Server thông qua địa chỉ URL. Sau khi nhận được yêu cầu, Server tiến hành xử lý và gửi phản hồi (Response) trở lại cho Client. Dữ liệu trao đổi giữa hai bên có thể ở nhiều định dạng khác nhau như HTML, JSON hoặc XML. HTTP sử dụng giao thức TCP/IP làm nền tảng truyền tải dữ liệu và hoạt động theo cơ chế Stateless, nghĩa là mỗi yêu cầu được xử lý độc lập mà không lưu trạng thái của các phiên làm việc trước đó.

Đặc điểm của giao thức HTTP: Giao thức HTTP có khả năng tương thích cao với nhiều nền tảng và thiết bị khác nhau. HTTP hỗ trợ nhiều phương thức truyền dữ liệu như GET, POST, PUT và DELETE, phù hợp với các ứng dụng Web và API hiện đại. Nhờ cấu trúc đơn giản và dễ triển khai, HTTP được sử dụng rộng rãi trong các hệ thống IoT để truyền dữ liệu cảm biến lên máy chủ hoặc nhận lệnh điều khiển từ hệ thống quản lý. Bên cạnh đó, HTTP hỗ trợ truyền dữ liệu dưới nhiều định dạng khác nhau và có khả năng tích hợp dễ dàng với các dịch vụ điện toán đám mây.

2.6.4 Chuẩn truyền thông HDMI

Chuẩn giao tiếp HDMI (High-Definition Multimedia Interface) là chuẩn truyền dẫn tín hiệu số được sử dụng để truyền đồng thời dữ liệu hình ảnh và âm thanh chất lượng cao giữa các thiết bị điện tử. Được phát triển vào năm 2002, hiện nay HDMI được sử dụng phổ biến trên máy tính, màn hình, tivi, camera và các hệ thống nhúng có nhu cầu hiển thị hình ảnh độ phân giải cao.

Nguyên tắc hoạt động: HDMI hoạt động bằng cách truyền dữ liệu số từ thiết bị phát

(Source) đến thiết bị nhận (Sink) thông qua cáp HDMI chuyên dụng. Tín hiệu hình ảnh và âm thanh được mã hóa và truyền trên các kênh dữ liệu tốc độ cao, giúp đảm bảo chất lượng tín hiệu trong quá trình truyền dẫn. Thiết bị nhận sẽ giải mã tín hiệu nhận được và hiển thị nội dung trên màn hình hoặc phát ra hệ thống âm thanh. Ngoài việc truyền dữ liệu đa phương tiện, HDMI còn hỗ trợ trao đổi thông tin điều khiển và nhận dạng thiết bị giữa hai đầu kết nối.

Đặc điểm của chuẩn HDMI: HDMI cho phép truyền đồng thời tín hiệu hình ảnh và âm thanh trên cùng một cáp kết nối, giúp giảm số lượng dây dẫn trong hệ thống. Chuẩn này hỗ trợ nhiều độ phân giải từ Full HD đến 4K và 8K tùy theo phiên bản. HDMI sử dụng phương thức truyền tín hiệu số nên hạn chế được nhiễu và suy hao tín hiệu so với các chuẩn tương tự. Ngoài ra, HDMI hỗ trợ âm thanh đa kênh chất lượng cao, khả năng tự động nhận diện thiết bị và tương thích với nhiều loại thiết bị điện tử hiện đại.

2.6.5 Chuẩn truyền thông USB HID

USB Human Interface Device (HID) là giao thức truyền thông chuẩn thuộc hệ sinh thái USB, được xây dựng nhằm hỗ trợ các thiết bị giao tiếp giữa con người và máy tính. Trong hệ thống được thiết kế, màn hình cảm ứng Waveshare 7 inch sử dụng chuẩn USB HID để gửi dữ liệu cảm ứng đến Raspberry Pi 4. Hệ điều hành Raspberry Pi OS hỗ trợ nhận diện tự động giao thức này mà không cần cài đặt thêm trình điều khiển. Việc sử dụng USB HID giúp quá trình tích hợp phần cứng trở nên đơn giản, đồng thời cho phép người dùng tương tác trực tiếp với giao diện hiển thị phục vụ công tác giám sát và chẩn đoán lỗi động cơ xe điện.

2.7 Thiết bị phần cứng

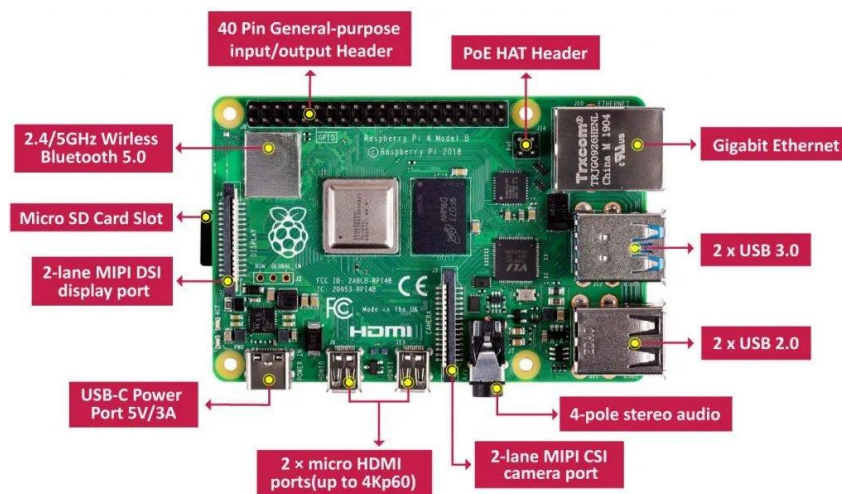
2.7.1 Máy tính Raspberry Pi 4

Raspberry Pi là dòng máy tính bo mạch đơn nhỏ gọn, kích thước chỉ bằng thẻ tín dụng, được phát triển tại Anh bởi Raspberry Pi Foundation nhằm thúc đẩy giáo dục khoa học máy tính. Đây là máy tính mini giá rẻ, chạy hệ điều hành Linux (như Raspberry Pi OS),

tích hợp đầy đủ tính năng, cổng kết nối (USB, HDMI, Wi-Fi, Ethernet, GPIO) và được dùng phổ biến cho các dự án DIY, IoT,...

Raspberry Pi 4, hoạt động tương tự như một máy tính hoàn chỉnh. Khi cấp nguồn, hệ thống sẽ khởi động từ thẻ nhớ microSD chứa hệ điều hành (thường là Linux như Raspberry Pi OS hoặc Ubuntu). Bộ xử lý trung tâm sẽ thực thi các lệnh chương trình, trong khi RAM lưu trữ dữ liệu tạm thời để xử lý. Người dùng có thể tương tác với hệ thống thông qua màn hình (HDMI), bàn phím, chuột hoặc truy cập từ xa qua mạng. Raspberry Pi có thể đóng vai trò như một bộ điều khiển trung tâm, giao tiếp với các thiết bị ngoại vi thông qua nhiều chuẩn truyền thông như GPIO, I2C, SPI, UART. Các chân GPIO cho phép đọc tín hiệu từ cảm biến và điều khiển thiết bị như LED, relay, motor (output). Ngoài ra, các giao diện như USB, Ethernet, Wi-Fi và Bluetooth giúp Raspberry Pi kết nối với mạng và các thiết bị khác, từ đó xây dựng các hệ thống IoT hoàn chỉnh.

Bảng mạch Raspberry Pi 4 được mô tả trong Hình 4.11. Với các thành phần sau:



Hình 2.2: Bảng mạch Raspberry Pi4

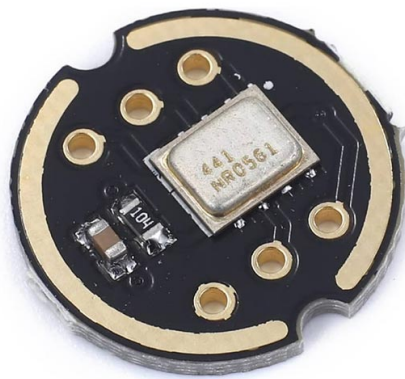
- 40 Pin General-purpose Input/Output Header (GPIO Header): Đây là cụm chân GPIO gồm 40 chân đa chức năng, cho phép Raspberry Pi giao tiếp với các thiết bị ngoại vi như cảm biến, màn hình, module truyền thông và các mạch điều khiển khác. Ngoài chức năng xuất nhập tín hiệu số, các chân này còn hỗ trợ nhiều giao thức như UART, SPI, I2C và PWM, giúp mở rộng khả năng kết nối phần cứng của hệ thống.
- PoE HAT Header: là cổng dùng để kết nối với module PoE HAT. Module này

cho phép Raspberry Pi nhận nguồn điện trực tiếp thông qua dây mạng Ethernet thay vì sử dụng bộ nguồn riêng. Tính năng này giúp giảm số lượng dây kết nối và thuận tiện trong các hệ thống đặt ở vị trí khó cấp nguồn.

- **Gigabit Ethernet:** Đây là cổng mạng Ethernet tốc độ Gigabit hỗ trợ truyền dữ liệu tốc độ cao lên đến 1000 Mbps. Cổng này được sử dụng để kết nối Raspberry Pi với mạng LAN hoặc Internet nhằm phục vụ truyền dữ liệu, điều khiển từ xa và kết nối với máy chủ.
- **Cổng USB 3.0:** Raspberry Pi 4 được trang bị hai cổng USB 3.0 tốc độ cao, hỗ trợ tốc độ truyền dữ liệu lên đến 5 Gbps. Các cổng này thường được sử dụng để kết nối ổ cứng ngoài, USB lưu trữ hoặc các thiết bị yêu cầu tốc độ truyền dữ liệu lớn.
- **USB 2.0:** Ngoài USB 3.0, Raspberry Pi 4 còn có hai cổng USB 2.0 hỗ trợ kết nối chuột, bàn phím, cảm biến hoặc các thiết bị ngoại vi thông thường. Tốc độ truyền dữ liệu tối đa của chuẩn USB 2.0 là 480 Mbps.
- **4-pole Stereo Audio:** Đây là cổng âm thanh chuẩn 3.5 mm tích hợp tín hiệu âm thanh và video analog. Cổng này cho phép kết nối loa, tai nghe hoặc các thiết bị âm thanh ngoài.
- **MIPI CSI Camera Port:** dùng để kết nối camera chuyên dụng dành cho Raspberry Pi. Chuẩn giao tiếp này có tốc độ truyền dữ liệu cao, thường được sử dụng trong các ứng dụng xử lý ảnh, thị giác máy tính và trí tuệ nhân tạo.
- **Micro HDMI Ports (up to 4Kp60):** Raspberry Pi 4 được trang bị hai cổng Micro HDMI hỗ trợ xuất hình ảnh đồng thời ra hai màn hình với độ phân giải tối đa lên đến 4K ở tần số quét 60Hz. Điều này giúp thiết bị đáp ứng tốt các nhu cầu hiển thị và xử lý đồ họa.
- **USB-C Power Port 5V/3A:** Đây là cổng cấp nguồn chính của Raspberry Pi 4 sử dụng chuẩn USB Type-C. Thiết bị yêu cầu nguồn điện 5V–3A để đảm bảo hoạt động ổn định, đặc biệt khi sử dụng nhiều thiết bị ngoại vi cùng lúc.
- **MIPI DSI Display Port:** là cổng kết nối dành cho màn hình hiển thị chuyên dụng của Raspberry Pi. Chuẩn giao tiếp này hỗ trợ tốc độ truyền dữ liệu cao và thường được dùng với màn hình LCD hoặc màn hình cảm ứng.

- Micro SD Card Slot: Khe cắm thẻ Micro SD là nơi lưu trữ hệ điều hành và dữ liệu của Raspberry Pi.
- 2.4/5GHz Wireless Bluetooth 5.0: Raspberry Pi 4 tích hợp sẵn WiFi băng tần kép 2.4GHz và 5GHz cùng Bluetooth 5.0. Tính năng này giúp thiết bị kết nối không dây với mạng Internet hoặc giao tiếp với các thiết bị ngoại vi như cảm biến, điện thoại hoặc thiết bị IoT khác.

2.7.2 Cảm biến âm thanh INMP441



Hình 2.3: Cảm biến âm thanh INMP441

INMP441 (Hình 2.3) là một cảm biến âm thanh kỹ thuật số sử dụng giao thức truyền thông I2S để thu nhận và truyền dữ liệu âm thanh. Cảm biến này được phát triển bởi InvenSense và được sử dụng phổ biến trong các ứng dụng như nhận diện giọng nói, IoT và xử lý tín hiệu âm thanh. Cảm biến INMP441 có thông số kỹ thuật được ghi ở Bảng 2.1.

Bảng 2.1: Thông số kỹ thuật của cảm biến INMP441

Thông số	Giá trị
Loại	Micro MEMS đa hướng
Giao tiếp	I2S Digital Output
Độ nhạy	-26 dBFS
Tỷ lệ tín hiệu trên nhiễu (SNR)	61 dBA
Dải tần đáp ứng	60 Hz – 15 kHz
Dải động (Dynamic Range)	87 dB
Độ méo hài tổng (THD)	3%
Mức âm thanh đầu vào tối đa	120 dB SPL
Điện áp hoạt động	1.62V – 3.63V
Dòng tiêu thụ hoạt động	1.4 mA @1.8V
Độ phân giải dữ liệu	24-bit
Kích thước	$4.72 \times 3.76 \times 1$ mm
Nhiệt độ hoạt động	-40°C đến +85°C

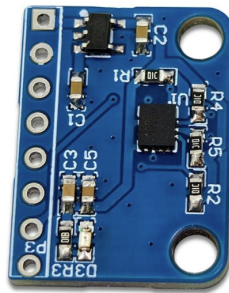
Nguyên tắc hoạt động: INMP441 hoạt động bằng cách chuyển đổi sóng âm thanh trong môi trường thành tín hiệu điện thông qua cấu trúc Micro-Electro-Mechanical Systems (MEMS). Sau đó, tín hiệu analog này được tích hợp sẵn bộ ADC bên trong để chuyển đổi thành tín hiệu số. Dữ liệu âm thanh số sẽ được truyền ra ngoài thông qua giao thức I2S. Cảm biến sử dụng các chân tín hiệu chính như: SD (Serial Data) để truyền dữ liệu âm thanh, SCK/BCLK (Bit Clock) để đồng bộ dữ liệu theo từng bit, và Word Select (WS/LRCLK) để xác định kênh âm thanh trái hoặc phải. INMP441 thường hoạt động như một thiết bị Slave, nhận xung clock từ vi điều khiển hoặc máy tính như Raspberry Pi đóng vai trò Master. Ngoài ra, chân L/R cho phép cấu hình microphone hoạt động ở kênh trái hoặc phải trong hệ thống stereo.

Đặc điểm của cảm biến INMP441: Tín hiệu đầu ra số, không cần mạch ADC ngoài, giúp giảm nhiễu và đơn giản hóa thiết kế phần cứng. Chất lượng âm thanh cao: Hỗ trợ độ phân giải 24-bit và dải tần rộng, phù hợp cho các ứng dụng xử lý âm thanh. Giao tiếp I2S: Truyền dữ liệu ổn định, đồng bộ tốt với vi điều khiển. Kích thước nhỏ gọn: Công

nghe MEMS giúp cảm biến có kích thước nhỏ, dễ tích hợp. Tiêu thụ điện năng thấp: Phù hợp cho các hệ thống IoT hoặc thiết bị chạy pin. Khả năng chống nhiễu tốt: Do sử dụng tín hiệu số thay vì analog.

Ứng dụng của INMP441: Cảm biến này được sử dụng rộng rãi trong các hệ thống nhận diện giọng nói, trợ lý ảo, thiết bị ghi âm, hệ thống giám sát âm thanh và các dự án AI xử lý tín hiệu. Trong các hệ thống IoT, INMP441 thường được kết hợp với vi điều khiển hoặc Raspberry Pi để thu thập dữ liệu âm thanh và gửi lên server để phân tích. Ngoài ra, nó còn được ứng dụng trong các thiết bị như loa thông minh, camera an ninh có thu âm và hệ thống điều khiển bằng giọng nói. . .

2.7.3 Cảm biến gia tốc ICM42688



Hình 2.4: Cảm biến ICM42688

ICM42688 (Hình 2.4) là cảm biến MEMS 6 bậc tự do (6-axis IMU – Inertial Measurement Unit) được phát triển bởi TDK InvenSense. Cảm biến tích hợp cảm biến gia tốc 3 trục và con quay hồi chuyển 3 trục trong cùng một chip, cho phép đo chuyển động tuyến tính và chuyển động quay với độ chính xác cao. Cảm biến ICM42688 có thông số kỹ thuật được ghi ở Bảng 2.2.

Bảng 2.2: Thông số kỹ thuật của cảm biến ICM42688

Thông số	Giá trị
Loại	IMU MEMS 6 trục (3-axis Accelerometer + 3-axis Gyroscope)
Giao tiếp	SPI, I2C, I3C
Dải đo gia tốc	$\pm 2g, \pm 4g, \pm 8g, \pm 16g$
Dải đo con quay hồi chuyển	± 15.625 đến ± 2000 dps
Độ phân giải ADC	16-bit
Tần số lấy mẫu tối đa	32 kHz
Điện áp hoạt động	1.71V – 3.6V
Dòng tiêu thụ gia tốc	0.65 mA
Dòng tiêu thụ Gyroscope	0.88 mA
Chế độ tiết kiệm điện	Hỗ trợ Low-power Mode
FIFO tích hợp	2 KB
Nhiệt độ hoạt động	-40°C đến +85°C
Kích thước	$2.5 \times 3 \times 0.86$ mm

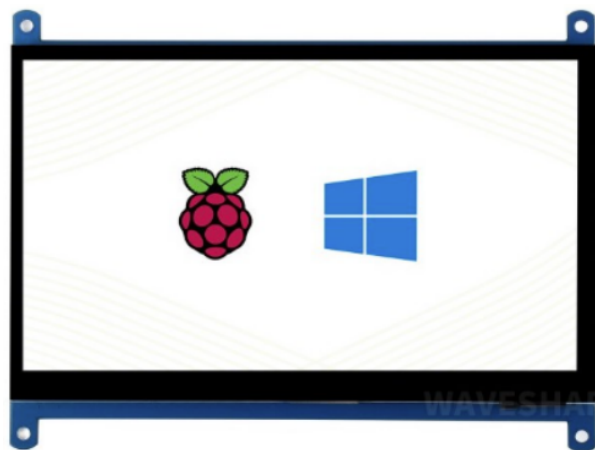
Nguyên tắc hoạt động: ICM42688 hoạt động dựa trên công nghệ MEMS (Micro-Electro-Mechanical Systems). Đối với cảm biến gia tốc, các cấu trúc vi cơ bên trong sẽ thay đổi vị trí khi xuất hiện gia tốc tác động, từ đó tạo ra sự thay đổi điện dung và được chuyển đổi thành tín hiệu điện. Đối với con quay hồi chuyển, cảm biến sử dụng hiệu ứng Coriolis để phát hiện vận tốc góc trên ba trục X, Y và Z. Các tín hiệu tương tự thu được sẽ được chuyển đổi sang dạng số thông qua bộ ADC tích hợp bên trong và truyền đến bộ xử lý thông qua giao tiếp SPI hoặc I2C. Cảm biến còn tích hợp bộ nhớ FIFO để lưu trữ dữ liệu tạm thời, giúp giảm tải quá trình đọc dữ liệu liên tục từ vi điều khiển.

Đặc điểm của cảm biến ICM42688: Tích hợp đồng thời cảm biến gia tốc và con quay hồi chuyển 3 trục giúp giảm kích thước hệ thống. Hỗ trợ nhiều chuẩn giao tiếp như SPI, I2C và I3C giúp dễ dàng tích hợp với các vi điều khiển và máy tính nhúng. Cảm biến có độ chính xác cao, tốc độ lấy mẫu lớn và độ trễ thấp, phù hợp với các ứng dụng xử lý tín hiệu thời gian thực. Công suất tiêu thụ thấp hỗ trợ chế độ tiết kiệm năng lượng, phù

hợp với các hệ thống IoT hoặc thiết bị sử dụng pin. Ngoài ra, bộ nhớ FIFO tích hợp giúp giảm tải cho bộ xử lý trung tâm và nâng cao hiệu quả thu thập dữ liệu.

Ứng dụng của ICM42688: Cảm biến được sử dụng rộng rãi trong các hệ thống theo dõi chuyển động, robot tự hành, thiết bị đeo thông minh, drone, hệ thống định vị quán tính và các ứng dụng công nghiệp. Trong đề tài này, ICM42688 được sử dụng để thu nhận tín hiệu rung động từ động cơ xe điện. Dữ liệu gia tốc thu được sẽ được xử lý, phân tích và kết hợp với dữ liệu âm thanh để phục vụ cho quá trình phát hiện và chẩn đoán lỗi động cơ bằng các thuật toán học máy.

2.7.4 Màn hình Capacitive Touch Screen LCD 7 inches



Hình 2.5: Màn hình Waveshare 7inch HDMI LCD (C)

Màn hình Waveshare 7inch HDMI LCD (C) (Hình 2.5) là màn hình cảm ứng điện dung kích thước 7 inch được thiết kế dành cho Raspberry Pi và các hệ thống nhúng. Màn hình sử dụng giao tiếp HDMI để truyền tín hiệu hình ảnh và giao tiếp USB để truyền dữ liệu cảm ứng. Với độ phân giải 1024×600 pixel cùng công nghệ tấm nền IPS, màn hình cung cấp chất lượng hiển thị tốt, góc nhìn rộng và khả năng tương tác trực tiếp với người dùng. Trong đề tài này, màn hình được sử dụng để hiển thị giao diện EV Guard Dashboard, cho phép theo dõi trạng thái hoạt động của động cơ xe điện theo thời gian thực.

Bảng 2.3: Thông số kỹ thuật của màn hình Waveshare 7inch HDMI LCD (C)

Thông số	Giá trị
Kích thước màn hình	7 inch
Loại màn hình	IPS LCD
Độ phân giải	1024 × 600 pixel
Giao tiếp hiển thị	HDMI
Giao tiếp cảm ứng	USB
Loại cảm ứng	Cảm ứng điện dung đa điểm
Góc nhìn	170°
Độ sáng	300 cd/m ²
Điện áp hoạt động	5V
Công suất tiêu thụ	Khoảng 3 W
Hỗ trợ hệ điều hành	Raspberry Pi OS, Ubuntu, Windows
Kích thước ngoài	164.9 × 124.27 mm

Nguyên tắc hoạt động: Màn hình sử dụng giao tiếp HDMI để nhận tín hiệu hình ảnh từ Raspberry Pi 4. Tín hiệu này được xử lý bởi bộ điều khiển tích hợp trên màn hình và hiển thị lên tấm nền LCD. Đồng thời, lớp cảm ứng điện dung phía trước màn hình có khả năng phát hiện vị trí chạm của người dùng. Dữ liệu cảm ứng được truyền về Raspberry Pi thông qua giao tiếp USB, cho phép thực hiện các thao tác điều khiển trực tiếp trên giao diện người dùng.

Đặc điểm của màn hình Waveshare 7inch HDMI LCD (C): Màn hình có kích thước phù hợp cho các hệ thống nhúng và thiết bị IoT. Công nghệ IPS cung cấp góc nhìn rộng và chất lượng hiển thị ổn định. Màn hình hỗ trợ cảm ứng điện dung đa điểm giúp nâng cao khả năng tương tác với người dùng. Giao tiếp HDMI và USB giúp việc kết nối với Raspberry Pi trở nên đơn giản mà không cần thiết kế thêm mạch điều khiển chuyên dụng. Ngoài ra, màn hình tương thích với nhiều hệ điều hành phổ biến và không yêu cầu cài đặt trình điều khiển phức tạp.

Ứng dụng của màn hình Waveshare 7inch HDMI LCD (C): Màn hình được sử dụng rộng rãi trong các hệ thống nhúng, thiết bị IoT, máy tính công nghiệp, hệ thống điều

khuyến tự động và các thiết bị giám sát. Trong đề tài này, màn hình đóng vai trò là giao diện người dùng của hệ thống chẩn đoán lỗi động cơ xe điện, hiển thị kết quả phân loại, độ tin cậy dự đoán, lịch sử cảnh báo, thống kê hoạt động và trạng thái cập nhật mô hình OTA. Việc tích hợp màn hình trực tiếp trên thiết bị giúp hệ thống có khả năng hoạt động độc lập mà không cần kết nối với máy tính bên ngoài.

Chương 3

Thiết kế và thi công hệ thống

3.1 Yêu cầu thiết kế

3.1.1 Yêu cầu về tính năng của sản phẩm

Mô hình hệ thống được thiết kế dựa trên yêu cầu của một hệ thống chẩn đoán lỗi động cơ xe điện và phải đáp ứng các yêu cầu sau:

- Hệ thống phải có khả năng thu thập dữ liệu rung động và âm thanh liên tục từ động cơ xe điện trong quá trình vận hành.
- Hệ thống phải thực hiện tiền xử lý dữ liệu, trích xuất đặc trưng và đưa dữ liệu vào mô hình học sâu để thực hiện quá trình chẩn đoán lỗi.
- Hệ thống phải có khả năng nhận dạng và phân loại trạng thái hoạt động của động cơ như trạng thái bình thường hoặc các dạng lỗi đã được huấn luyện trước.
- Kết quả chẩn đoán cần được hiển thị trực tiếp trên giao diện người dùng để hỗ trợ quá trình theo dõi và giám sát.
- Hệ thống cho phép cập nhật thuật toán xử lý hoặc mô hình học sâu từ xa thông qua OTA.
- Người dùng có thể theo dõi dữ liệu cảm biến, trạng thái hoạt động của hệ thống và kết quả chẩn đoán thông qua hoặc màn hình hiển thị.

3.1.2 Yêu cầu về kỹ thuật

Chức năng kỹ thuật

- Thu thập dữ liệu rung động từ cảm biến ICM42688 và dữ liệu âm thanh từ cảm biến INMP441.
- Thực hiện phân loại và chẩn đoán lỗi động cơ theo thời gian thực.
- Hiển thị kết quả chẩn đoán và trạng thái hệ thống trên giao diện người dùng.
- Hỗ trợ cập nhật phần mềm từ xa thông qua OTA Server.

Thông số kỹ thuật

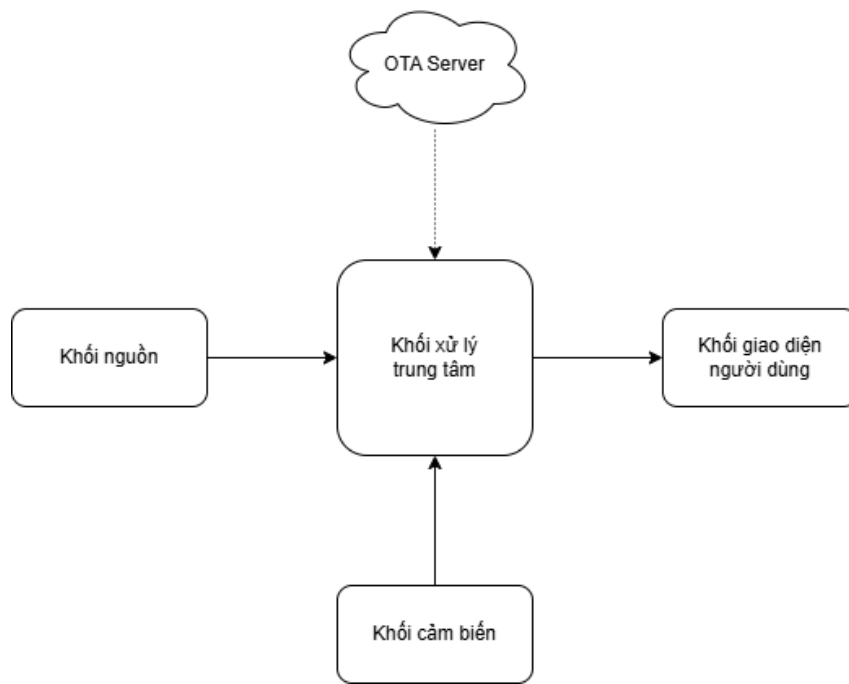
- Hệ thống sử dụng nguồn cấp 5V DC.
- Kích thước: xấp xỉ 20 x 15 x 15 (cm)
- Thời gian khởi động hệ thống sau khi cấp nguồn khoảng 10–20 giây.
- Hệ thống hỗ trợ cập nhật phần mềm OTA thông qua kết nối WiFi.
- Thời gian xử lý và hiển thị kết quả chẩn đoán đáp ứng yêu cầu thời gian thực.

3.2 Đặc tả hệ thống

Hệ thống được xây dựng bao gồm 5 khối chính kết nối và tương tác với nhau như Hình 3.1:

- Khối cảm biến: Khối này có nhiệm vụ thu thập các dữ liệu từ động cơ xe điện trong quá trình vận hành. Dữ liệu thu nhận bao gồm tín hiệu rung động và tín hiệu âm thanh, phản ánh trạng thái hoạt động của động cơ. Các dữ liệu này được sử dụng làm đầu vào cho quá trình phân tích và chẩn đoán lỗi.

- Khối xử lý trung tâm: Đây là khối đảm nhiệm vai trò xử lý và điều phối toàn bộ hoạt động của hệ thống. Khối xử lý trung tâm tiếp nhận dữ liệu từ khối cảm biến, thực hiện các bước xử lý dữ liệu, trích xuất đặc trưng và thực hiện thuật toán chẩn đoán lỗi. Đồng thời khối này cũng gửi kết quả xử lý đến khối giao diện người dùng và hỗ trợ giao tiếp với máy chủ cập nhật từ xa.



Hình 3.1: Sơ đồ khối hệ thống

- Khối giao diện người dùng: Khối này có nhiệm vụ hiển thị trạng thái hoạt động của hệ thống, dữ liệu thu thập được và kết quả chẩn đoán lỗi. Giao diện cho phép người dùng dễ dàng quan sát, theo dõi và tương tác với hệ thống trong quá trình vận hành.

- Khối nguồn: Là khối cung cấp điện áp phù hợp cho toàn bộ hệ thống hoạt động ổn định. Khối nguồn đảm bảo các thành phần như cảm biến, bộ xử lý trung tâm và các thiết bị ngoại vi được cấp nguồn liên tục và an toàn.

- OTA Server: Máy chủ OTA (Over-The-Air) được sử dụng để hỗ trợ cập nhật phần mềm từ xa cho hệ thống thông qua kết nối mạng. Thông qua chức năng này, các thuật toán xử lý, chương trình điều khiển hoặc mô hình học sâu có thể được nâng cấp mà không cần thao tác trực tiếp trên thiết bị.

Các khối trong hệ thống phối hợp với nhau để tạo thành một hệ thống chẩn đoán lỗi động cơ xe điện hoàn chỉnh. Dữ liệu từ khối cảm biến được truyền đến khối xử lý trung tâm để phân tích và đưa ra kết quả chẩn đoán. Sau đó, kết quả được hiển thị đến người dùng thông qua giao diện và hệ thống có thể được cập nhật thông qua máy chủ OTA khi cần thiết.

3.3 Thiết kế phần cứng

Hệ thống được thiết kế bao gồm các thành phần chính: khối nguồn, khối cảm biến, khối xử lý trung tâm, khối giao diện người dùng và máy chủ OTA. Mỗi khối đảm nhận một chức năng riêng và phối hợp với nhau nhằm thực hiện quá trình thu thập dữ liệu, xử lý tín hiệu, chẩn đoán lỗi và hiển thị kết quả.

3.3.1 Khối nguồn

Khối nguồn có nhiệm vụ cung cấp điện áp ổn định cho toàn bộ hệ thống hoạt động. Trong hệ thống này, các thiết bị như Raspberry Pi 4, màn hình cảm ứng Waveshare 7 inch, cảm biến ICM42688 và cảm biến INMP441 đều sử dụng nguồn điện áp thấp DC. Do đó, cần lựa chọn bộ nguồn phù hợp nhằm đảm bảo các thiết bị hoạt động ổn định, tránh hiện tượng sụt áp hoặc thiếu dòng trong quá trình hệ thống vận hành.

Các thiết bị trong hệ thống chủ yếu sử dụng nguồn điện áp 5V. Riêng cảm biến ICM42688 và INMP441 hoạt động ở mức điện áp thấp hơn nhưng được cấp thông qua các mạch chuyển mức hoặc bộ ổn áp trên module cảm biến. Vì vậy, hệ thống sử dụng một Adapter nguồn 5V làm nguồn cấp chính.

Tính toán dòng điện cấp cho hệ thống:

- Raspberry Pi 4: Theo tài liệu của nhà sản xuất, Raspberry Pi 4 khuyến nghị sử dụng nguồn 5V–3A do thiết bị tích hợp WiFi, Bluetooth và thực hiện xử lý dữ liệu tương đối lớn. Trong quá trình chạy hệ điều hành, xử lý tín hiệu và thực hiện mô hình học sâu, dòng tiêu thụ thực tế có thể dao động khoảng từ 1.5A đến 2A.

- Màn hình cảm ứng Waveshare 7 inch HDMI LCD (C): Theo thông số từ nhà sản xuất, màn hình sử dụng nguồn 5V và dòng tiêu thụ khoảng 490mA khi bật đèn nền.

:contentReference[oaicite:1]index=1

- Cảm biến ICM42688: Cảm biến có mức tiêu thụ công suất thấp, dòng hoạt động khoảng 0.65mA ở chế độ gia tốc và khoảng 0.88mA ở chế độ con quay hồi chuyển. Tổng dòng tiêu thụ xấp xỉ:

$$I_{ICM42688} = 0.65 + 0.88$$

$$I_{ICM42688} = 1.53mA$$

- Cảm biến âm thanh INMP441: Dòng hoạt động của cảm biến khoảng 1.4mA.

Như vậy tổng dòng tiêu thụ của hệ thống được tính:

$$I_{Tong} = I_{RaspberryPi} + I_{LCD} + I_{ICM42688} + I_{INMP441}$$

Thay số:

$$I_{Tong} = 2000 + 490 + 1.53 + 1.4$$

$$I_{Tong} = 2492.93mA$$

$$I_{Tong} \approx 2.5A$$

Như vậy hệ thống cần tối thiểu khoảng 2.5A nguồn dòng và 5V nguồn áp để hoạt động ổn định. Tuy nhiên để dự phòng cho trường hợp phát sinh thêm thiết bị ngoại vi hoặc sai số trong quá trình tính toán, lựa chọn bộ nguồn Adapter 5V–3A là phù hợp. Bộ nguồn này có khả năng cung cấp dòng tối đa 3A và chỉ cấp đúng lượng dòng theo nhu cầu tải của hệ thống, giúp đảm bảo tính ổn định và tăng độ tin cậy trong quá trình vận hành.

3.3.2 Khối cảm biến

Khối cảm biến bao gồm cảm biến gia tốc ICM42688 và cảm biến âm thanh INMP441. Cảm biến ICM42688 thực hiện thu thập dữ liệu dao động rung của động cơ thông qua tín hiệu gia tốc ba trục, trong khi cảm biến INMP441 thu nhận tín hiệu âm thanh phát sinh trong quá trình hoạt động của động cơ. Hai loại dữ liệu này được sử dụng nhằm khai thác đồng thời thông tin rung động và âm thanh để nâng cao độ chính xác trong quá trình phát hiện lỗi.

Đối với tín hiệu rung, ICM42688 được lựa chọn do có tốc độ lấy mẫu 32kHz là rất cao, độ nhạy tốt, đáp ứng được tốc độ lấy mẫu của dataset sử dụng huấn luyện mô hình AI và tích hợp cảm biến gia tốc cùng con quay hồi chuyển trên một chip duy nhất. Điều này giúp thu nhận dữ liệu rung động với độ chính xác cao và giảm kích thước phần cứng. Ngoài ra, cảm biến hỗ trợ giao tiếp SPI tốc độ cao, phù hợp cho các ứng dụng yêu cầu thu thập dữ liệu thời gian thực.

Đối với tín hiệu âm thanh, INMP441 được lựa chọn do sử dụng chuẩn giao tiếp I2S với đầu ra số trực tiếp, không cần mạch ADC ngoài (điều này giúp đơn giản hóa thiết kế phần cứng), hoạt động tốt với âm thanh có tần số 60Hz đến 15kHz. Cảm biến có khả năng thu nhận âm thanh với độ phân giải cao, phù hợp cho các ứng dụng xử lý tín hiệu và phân tích đặc trưng âm học.

3.3.3 Khối xử lý trung tâm

Khối xử lý trung tâm sử dụng Raspberry Pi 4 đóng vai trò là bộ điều khiển chính của hệ thống. Khối này có nhiệm vụ nhận dữ liệu từ các cảm biến thông qua các giao thức truyền thông như SPI và I2S, thực hiện tiền xử lý tín hiệu, trích xuất đặc trưng và đưa dữ liệu vào mô hình học sâu để thực hiện quá trình nhận dạng và chẩn đoán lỗi động cơ. Ngoài ra, Raspberry Pi còn đảm nhiệm chức năng quản lý dữ liệu, điều khiển giao diện hiển thị và hỗ trợ cập nhật phần mềm từ xa.

Raspberry Pi 4 được lựa chọn do sở hữu bộ vi xử lý đa nhân với hiệu năng xử lý tương đối cao, hỗ trợ hệ điều hành Linux và tương thích với nhiều thư viện học máy như TensorFlow, PyTorch và Scikit-learn. So với các vi điều khiển thông thường, Raspberry Pi có khả năng xử lý dữ liệu lớn và triển khai các mô hình học sâu trực tiếp trên thiết bị. Ngoài ra, thiết bị tích hợp sẵn WiFi, Bluetooth và nhiều chuẩn giao tiếp như I2S, UART, SPI, I2C... giúp dễ dàng mở rộng hệ thống.

3.3.4 Khối giao diện người dùng

Khối giao diện người dùng cho phép hiển thị trạng thái hoạt động của hệ thống, dữ liệu cảm biến và kết quả chẩn đoán lỗi. Người dùng có thể theo dõi thông tin theo thời

gian thực và thực hiện thao tác điều khiển hoặc cấu hình hệ thống khi cần thiết.

Trong hệ thống, màn hình cảm ứng được lựa chọn nhằm nâng cao khả năng tương tác trực tiếp với người dùng. Việc sử dụng cảm ứng giúp quá trình giám sát, theo dõi và thao tác với hệ thống trở nên thuận tiện hơn so với phương pháp sử dụng các nút nhấn truyền thống.

3.3.5 Máy chủ OTA

Máy chủ OTA hỗ trợ cập nhật phần mềm từ xa thông qua kết nối mạng mà không cần can thiệp trực tiếp vào thiết bị. Chức năng này giúp dễ dàng nâng cấp thuật toán xử lý, cập nhật mô hình học sâu hoặc sửa lỗi phần mềm trong quá trình vận hành hệ thống.

Việc tích hợp OTA giúp giảm thời gian bảo trì hệ thống và tăng tính linh hoạt trong quá trình phát triển. Khi thuật toán xử lý hoặc mô hình học sâu được cải tiến, hệ thống có thể cập nhật phiên bản mới mà không cần tháo rời hoặc lập trình lại trực tiếp trên thiết bị.

Trong hệ thống này, GitHub được lựa chọn làm máy chủ lưu trữ cho quá trình cập nhật OTA. Lý do lựa chọn GitHub là nền tảng này cung cấp khả năng lưu trữ mã nguồn và tập tin miễn phí, đồng thời hỗ trợ quản lý phiên bản hiệu quả thông qua hệ thống Git. Việc sử dụng GitHub cho phép dễ dàng theo dõi lịch sử cập nhật, quản lý các phiên bản phần mềm và nhanh chóng triển khai các thay đổi mới. Ngoài ra, GitHub hỗ trợ truy cập dữ liệu thông qua các liên kết trực tiếp (raw file URL), giúp Raspberry Pi có thể tải xuống các tệp chương trình, mô hình học sâu hoặc các gói cập nhật một cách thuận tiện thông qua giao thức HTTP/HTTPS.

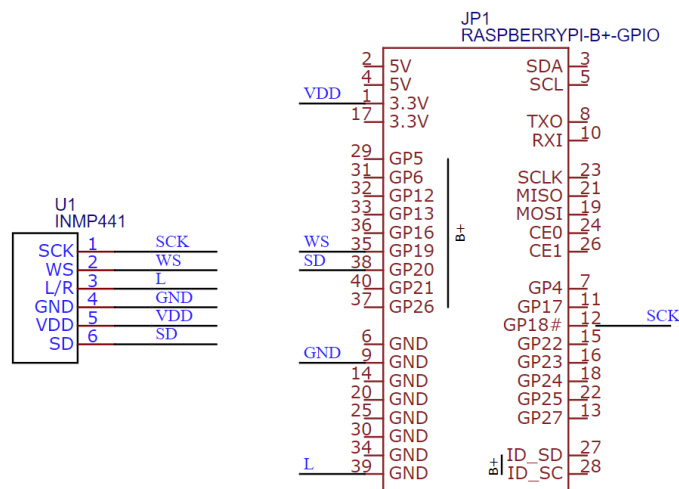
Như vậy, nguyên lý hoạt động của hệ thống được thực hiện như sau: Sau khi cấp nguồn, cảm biến ICM42688 và INMP441 tiến hành thu thập dữ liệu rung và âm thanh từ động cơ xe điện. Dữ liệu sau đó được truyền đến Raspberry Pi để thực hiện tiền xử lý và phân tích. Mô hình học sâu được triển khai trên Raspberry Pi sẽ xử lý dữ liệu đầu vào và đưa ra kết quả phân loại tình trạng hoạt động hoặc lỗi của động cơ. Kết quả sau cùng được hiển thị trên giao diện người dùng. Đồng thời, hệ thống có thể kết nối với máy chủ OTA để thực hiện cập nhật phần mềm khi cần thiết.

3.3.6 Sơ đồ nguyên lý

Cảm biến âm thanh INMP441 được kết nối với Raspberry Pi 4 thông qua giao tiếp I2S để thu thập dữ liệu âm thanh số. Nguồn cấp 3.3V được lấy trực tiếp từ Raspberry Pi. Các chân SCK, WS và SD lần lượt được sử dụng để truyền xung đồng hồ, tín hiệu đồng bộ kênh và dữ liệu âm thanh. Chân L/R được nối xuống mức thấp để cấu hình microphone hoạt động ở kênh trái (Left). Chi tiết kết nối được trình bày trong Bảng 3.1.

INMP441	Raspberry Pi4
VDD	3.3V – Pin 1
GND	GND – Pin 9
SCK	GPIO18 – Pin 12
WS	GPIO19 – Pin 35
SD	GPIO20 – Pin 38
L/R	Pin 39 (kênh Left)

Bảng 3.1: Kết nối INMP441 với Raspberry Pi 4

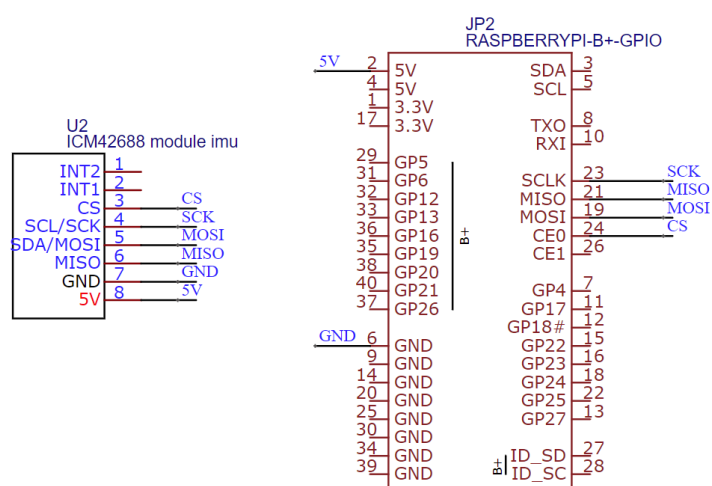


Hình 3.2: Sơ đồ kết nối Raspberry Pi và INMP441

Cảm biến gia tốc ICM-42688 được kết nối với Raspberry Pi 4 thông qua giao tiếp SPI nhằm đảm bảo tốc độ truyền dữ liệu cao và ổn định. Raspberry Pi đóng vai trò Master, thực hiện đọc dữ liệu gia tốc từ cảm biến thông qua các đường tín hiệu MOSI, MISO, SCLK và CS. Chi tiết kết nối được trình bày trong Bảng 3.2.

ICM-42688	Raspberry Pi4
VCC	3.3V – Pin 1
GND	GND – Pin 6
MOSI/SDA	GPIO10 – Pin 19
MISO/ADO	GPIO9 – Pin 21
SCLK/SCL	GPIO11 – Pin 23
CS/NCS	GPIO8 – Pin 24

Bảng 3.2: Kết nối cảm biến ICM-42688 với Raspberry Pi 4



Hình 3.3: Sơ đồ kết nối Raspberry Pi và ICM42688

3.4 Thiết kế phần mềm

3.4.1 Dữ liệu và tiền xử lý tín hiệu

Hệ thống hướng đến mục tiêu tối ưu hóa độ chính xác chẩn đoán thực tế và bảo đảm tính bền vững của dữ liệu thời gian thực khi vận hành trên nền tảng máy tính hạn chế về tài nguyên. Bởi vậy, quy trình thu thập đa nguồn và đồng bộ hóa hình học ma trận đặc trưng là bước quan trọng đầu tiên không thể thiếu.

Đặc tả tập dữ liệu và quy trình phân chia

Đề tài tích hợp dữ liệu đa nguồn để khai thác song song hai miền cảm biến biệt lập, với 2100 mẫu mô phỏng lại các kịch bản sự cố cơ khí trong môi trường vận hành công nghiệp:

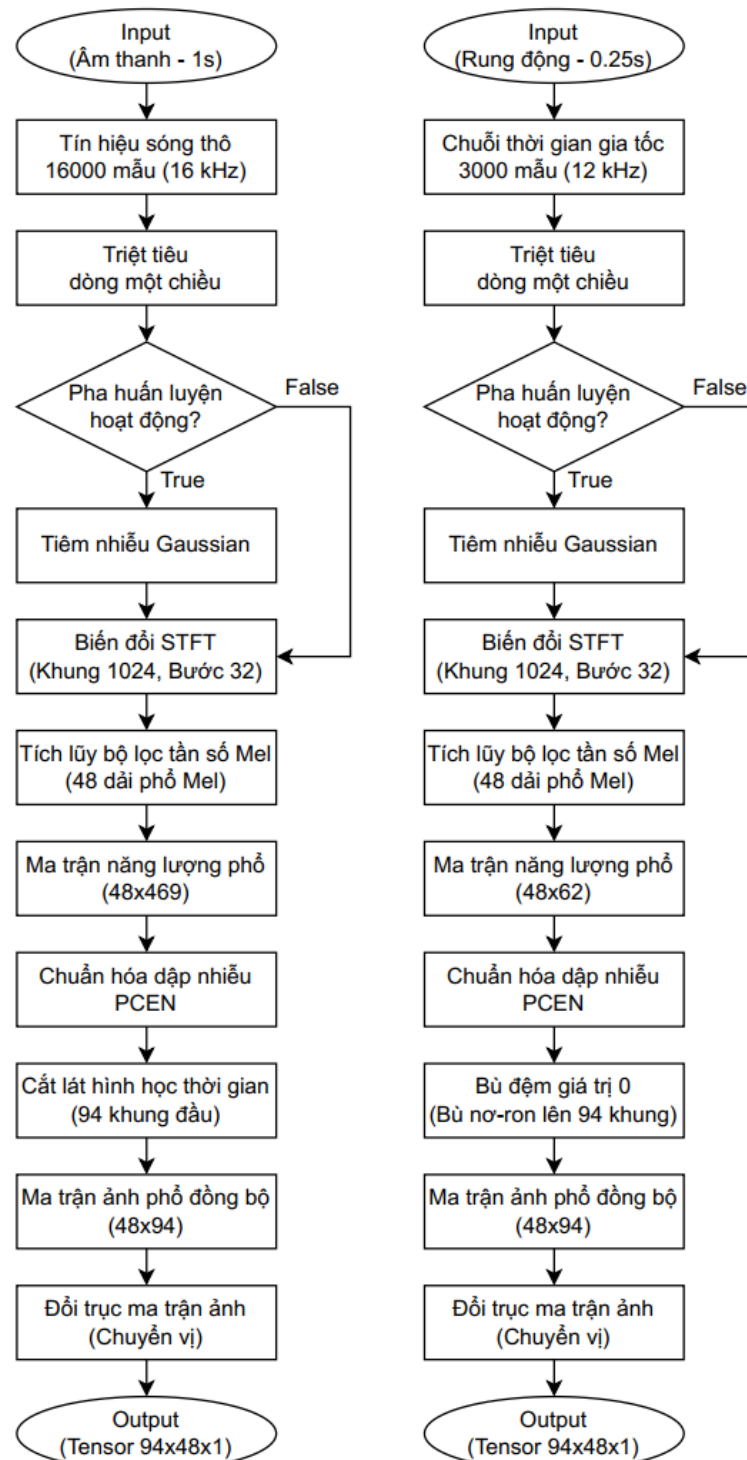
- Tập dữ liệu rung động: Được trích xuất từ tập dữ liệu chuẩn quốc tế của Đại học Case Western Reserve (CWRU). Dữ liệu thu thập từ cảm biến gia tốc đặt tại gối đỡ phía đầu trục động cơ với tần số lấy mẫu là 12.000 Hz. Cửa sổ thời gian thực thi được cố định ở mức 3.000 mẫu liên tục (khoảng 0,25 giây hoạt động thực tế). Môi trường thu thập dữ liệu gốc là hệ thống thử nghiệm tĩnh trong phòng thí nghiệm, nơi nhiều nền cơ khí được kiểm soát nghiêm ngặt, tập trung phản ánh chính xác các xung lực học nội tại phát sinh từ các vết nứt cấu trúc vòng bi (vòng trong, vòng ngoài, con lăn).
- Tập dữ liệu âm thanh: Được trích xuất từ tập dữ liệu IDMT-ISA-ELECTRIC-ENGINE. Tín hiệu áp suất sóng âm được thu nhận bằng microphone công nghiệp đặt trong phân xưởng sản xuất thực tế với tần số lấy mẫu chuẩn hóa là 16.000 Hz. Phân đoạn thời gian nạp vào được cấu hình dài 1,0 giây liên tục (khoảng 16.000 mẫu). Khác với nhánh rung động sạch, miền dữ liệu âm thanh này mang đặc tính suy hao cao và bị ô nhiễm nặng nề bởi các tạp âm nền ngẫu nhiên ngoài đời thực bao gồm: tiếng quạt thông gió công nghiệp, tiếng bước chân, và âm thanh cộng hưởng từ các thiết bị cơ khí lân cận trong nhà máy.

Sự kết hợp này tạo thành ma trận 6 nhãn trạng thái cân bằng được xếp từ 0 đến 5 bao gồm: Hoạt động bình thường (Normal), Vận hành quá tải (HeavyLoad), Lỗi vòng trong vòng bi (InnerRace), Lỗi vòng ngoài vòng bi (OuterRace), Lỗi con lăn (BallFault), và Hư hỏng phần cứng nặng (Broken).

Quy trình phân tách dữ liệu được thiết kế nghiêm ngặt từ file thô ban đầu, thay vì cắt đoạn ngẫu nhiên. Danh sách toàn bộ đường dẫn file thô được đưa vào khối bóc tách tên file để loại bỏ các phần tử trùng lặp thông qua cấu trúc tập hợp nhằm thu được mảng các tệp tin duy nhất. Thuật toán tiến hành phân chia ngẫu nhiên mảng tên file này với tỷ lệ cấu hình tập huấn luyện chiếm 80% và tập kiểm thử chiếm 20%. Các đường dẫn tệp tin gốc khớp với danh sách phân chia sẽ được gom về hai pool nguồn độc lập để phục vụ

cho các bước trích xuất tiếp theo.

Thuật toán trích xuất đặc trưng từng nhánh cảm biến



Hình 3.4: Lưu đồ thuật toán trích xuất đặc trưng từng nhánh cảm biến

Mặc dù cả hai nhánh dữ liệu đều được đồng bộ về cấu trúc ma trận kích thước $94 \times 48 \times 1$ trước khi nạp vào mạng học sâu, các bước biến đổi trên từng luồng tín hiệu số được thực hiện biệt lập dựa theo thuộc tính vật lý của cảm biến.

Đối với nhánh biến đổi tín hiệu rung động, đầu tiên, cắt một cửa sổ dữ liệu gồm chính xác 3000 mẫu liên tục từ chuỗi thời gian thô, tương ứng với khoảng 0.25 giây hoạt động với tần số lấy mẫu gốc là 12000 Hz. Hệ thống thực hiện phép trừ giá trị trung bình toán học trên toàn chuỗi mẫu để khử thành phần dòng một chiều, đưa biên độ dao động về điểm cân bằng ổn định 0.

Trong pha huấn luyện, kỹ thuật tăng cường dữ liệu được áp dụng bằng cách thêm nhiễu nhằm nâng cao tính tổng quát cho trọng số mô hình. Chuỗi dữ liệu tiếp tục được đưa qua phép biến đổi Fourier ngắn hạn (STFT) với kích thước khung ảnh phổ 1024 và bước nhảy thời gian 32, sau đó tích lũy năng lượng thông qua ma trận gồm 48 dải bộ lọc Mel. Tại bước này, số lượng khung thời gian trích xuất được từ bước trượt là 62 khung, tạo ra ma trận đặc trưng kích thước 48×62 .

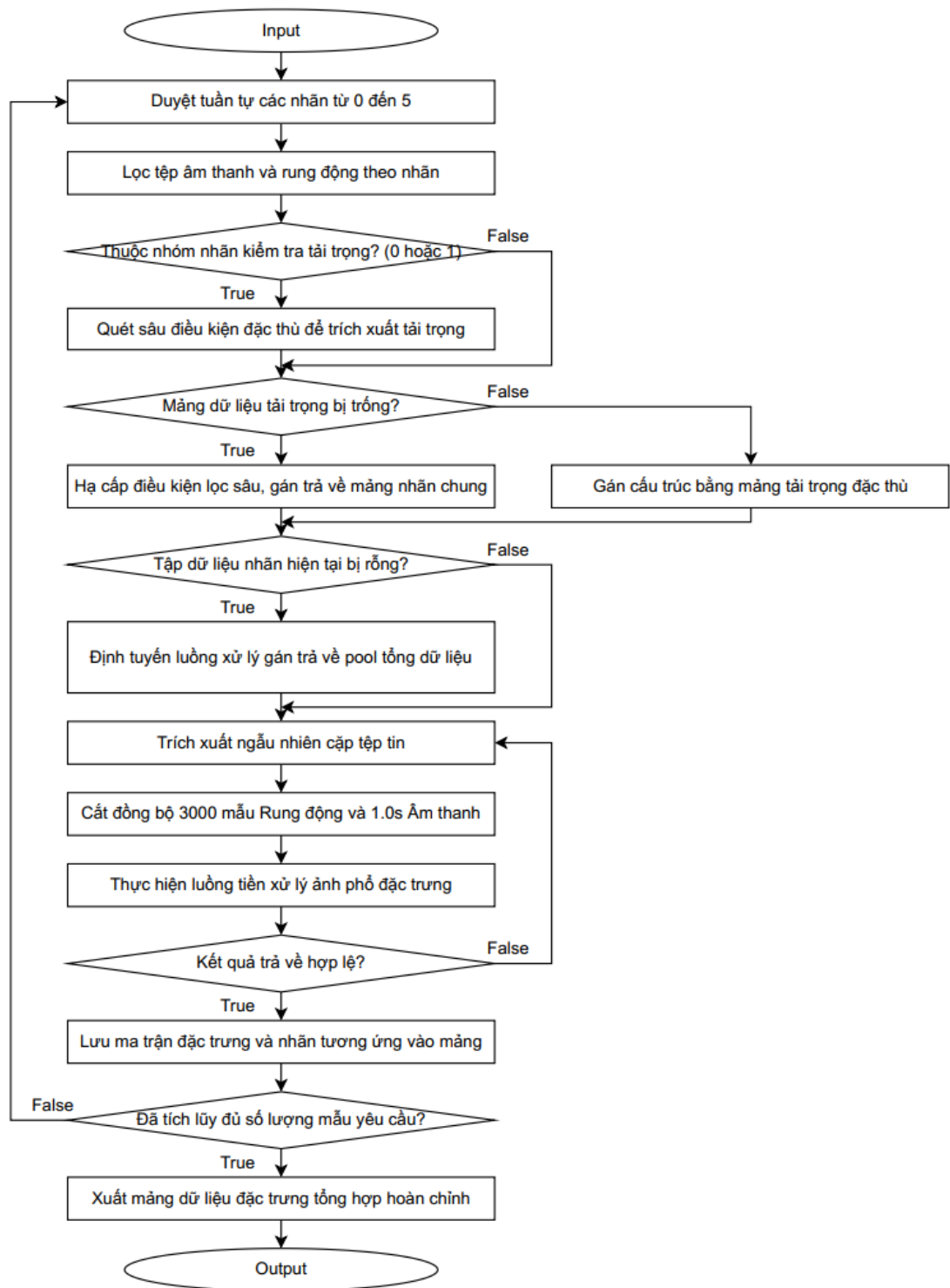
Cuối cùng, ma trận được dẫn truyền qua bộ lọc thích ứng PCEN để dập tắt các thành phần rung động điều hòa liên tục của trục máy và làm nổi bật các đỉnh xung va đập. Do số lượng khung thời gian hiện tại là 62 nhỏ hơn tham số cấu hình hệ thống là 94, giải thuật thực hiện bù đệm giá trị 0 (Zero-padding) vào phần đuôi của trục thời gian để kéo giãn ma trận đạt kích thước 48×94 , trước khi thực hiện phép chuyển vị đưa Tensor về đúng chiều hình học $94 \times 48 \times 1$.

Đối với nhánh biến đổi tín hiệu Âm thanh, tín hiệu áp suất sóng âm truyền trong không khí mang đặc tính suy hao cao, yêu cầu một quy trình đối xứng nhưng có biên độ bao phủ thời gian rộng hơn. Quá trình trích xuất phân đoạn thời gian nạp một phân đoạn sóng thô dài liên tục 1 giây, tương ứng với 16000 mẫu biên độ âm thanh tại tần số lấy mẫu đã chuẩn hóa là 16000 Hz. Chuỗi mẫu âm thanh được chuẩn hóa hiệu số trung bình nhằm triệt tiêu các sai số điện áp một chiều của phần cứng.

Tiếp theo, tín hiệu được phân tích cấu trúc phổ qua hàm STFT với cấu hình đối xứng gồm kích thước cửa sổ 1024 và bước trượt 32. Do chiều dài mẫu thô ban đầu lớn, số lượng khung thời gian trích xuất được sau cấu trúc trượt đạt đến 469 khung. Năng lượng phổ được ánh xạ qua ma trận 48 dải lọc Mel để thu được ma trận kích thước 48×469 .

Sau khi đi qua thuật toán Per-Channel Energy Normalization (PCEN) để sinh ra để trị các loại tạp âm nền ổn định, giải thuật thực hiện phương pháp cắt lát hình học do số lượng khung thời gian 469 vượt quá tham số cấu hình của mạng. Hệ thống chỉ bóc lấy đúng 94 khung thời gian đầu tiên của ảnh phổ và loại bỏ toàn bộ các khung thừa phía sau, thu được ma trận kích thước 48x94 trước khi thực hiện phép chuyển vị để xuất ra định dạng Tensor đồng bộ 94x48x1.

Cơ chế điều hướng dự phòng dữ liệu



Hình 3.5: Lưu đồ cơ chế điều hướng dự phòng dữ liệu

Sau khi định hình hai luồng tiền xử lý, phần mềm thiết lập tiến trình bốc mẫu tự động kết hợp cơ chế điều hướng dự phòng hai tầng nhằm ngăn chặn hoàn toàn lỗi rỗng mảng

dữ liệu gây sập chương trình khi vận hành thực tế tại biên.

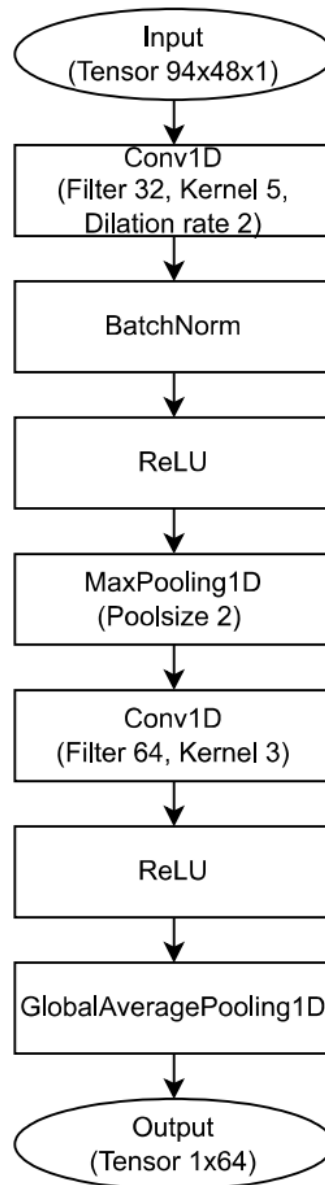
Giải thuật vận hành dựa trên một vòng lặp chính duyệt qua lần lượt 6 nhãn trạng thái. Tại mỗi vòng lặp nhãn, hệ thống tự động lọc sơ bộ tập âm thanh theo các từ khóa thư mục ứng với động cơ và tập rung động theo từ khóa mã lỗi cơ khí. Đối với hai nhãn đặc thù về tải trọng gồm Hoạt động Normal và HeavyLoad, một bộ lọc nâng cao sẽ quét sâu các ký tự hậu tố đặc trưng của tải trọng động cơ trong hệ thống để nạp vào mảng tạm.

Nếu điều kiện lọc sâu tải trọng này trả về kết quả rỗng do thiếu hụt tệp tin cục bộ trên bộ nhớ thiết bị biên, hệ thống tự động kích hoạt tầng dự phòng thứ nhất bằng cách hạ cấp điều kiện lọc và gán trả về pool nhãn chung của lỗi đó. Trường hợp pool nhãn chung hoặc pool âm thanh vẫn trống, tầng dự phòng thứ hai sẽ được kích hoạt để tự động gán trả về pool tổng của toàn bộ hệ thống. Cơ chế điều hướng tuần hoàn này đảm bảo luồng bốc mẫu ngẫu nhiên luôn tiếp cận được tệp tin hợp lệ để thực thi trích xuất cửa sổ đồng bộ 3000 mẫu rung động và 1.0 giây âm thanh, xuất ra mảng đặc trưng tổng hợp cân bằng cho cả 6 lớp trạng thái hoạt động của động cơ.

3.4.2 Kiến trúc mô hình học sâu

Kiến trúc mạng sâu của mô hình được định hình chi tiết qua 3 khối xử lý logic, phân cấp rõ ràng từ khâu trích xuất song lập, hợp nhất có điều kiện cho đến khối phân loại đầu ra, đảm bảo tối ưu hóa tính toán khi vận hành trên thiết bị nhúng.

Khối trích xuất đặc trưng song song



Hình 3.6: Lưu đồ khối trích xuất đặc trưng song song

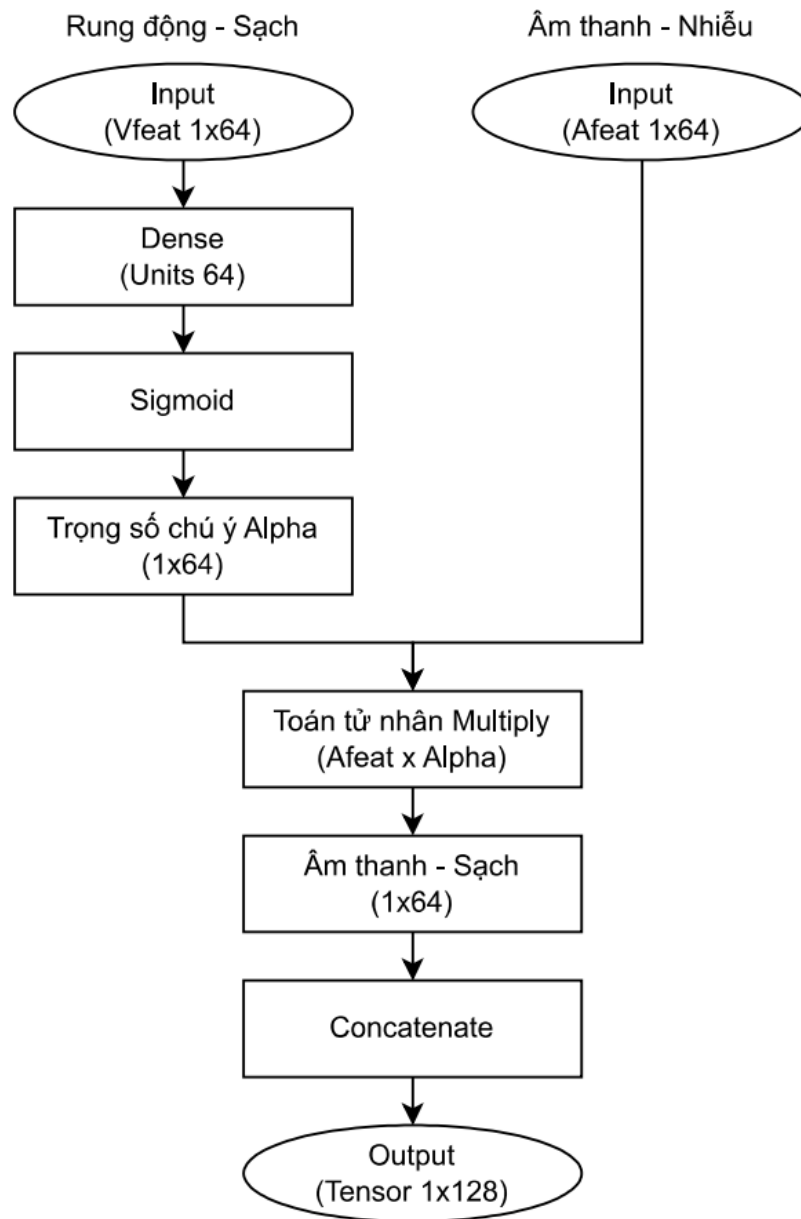
Khối trích xuất đặc trưng song song đảm nhiệm vai trò mã hóa các thuộc tính vật lý độc lập từ ma trận ảnh phổ đặc trưng của rung động và âm thanh. Tại tầng tích chập đầu tiên của cả hai nhánh, giải thuật áp dụng kỹ thuật tích chập giãn (Dilated Convolution). Khác với các tầng tích chập thông thường, việc tích hợp hệ số giãn giúp mạng nơ-ron mở rộng trường nhìn kiến trúc theo trục thời gian một cách hiệu quả mà không làm gia tăng số lượng tham số toán học hay làm suy giảm độ phân giải của ảnh phổ. Cơ chế này cho phép mô hình bao quát và liên kết chặt chẽ các đỉnh xung và đập cơ khí có khoảng cách

thời gian xa nhau do chu kỳ quay của trục động cơ tạo ra.

Ngay sau quá trình tích chập, dòng dữ liệu được dẫn truyền qua tầng chuẩn hóa (Batch Normalization) nhằm ổn định phân phối phân tầng của các trọng số nơ-ron, tăng tốc độ hội tụ và giảm sự phụ thuộc vào quá trình khởi tạo ngẫu nhiên ban đầu. Hàm kích hoạt phi tuyến ReLU được thiết lập ngay sau đó để thực hiện phép toán ép các giá trị biên độ âm về số 0.

Sau khi bản đồ đặc trưng được hạ mẫu thông qua tầng trích xuất giá trị biên độ lớn nhất (MaxPooling1D) để tạo tính bất biến đối với các dịch chuyển nhỏ của tín hiệu, dữ liệu tiếp tục đi qua một tầng tích chập phụ để trừu tượng hóa đặc trưng ở cấp độ cao hơn. Ở giai đoạn cuối của hai nhánh, thay vì ứng dụng lớp kéo phẳng ma trận (Flatten) truyền thống khiến dữ liệu được kéo dài thành một chuỗi nơ-ron đơn điệu gây bùng nổ hàng triệu tham số kết nối liên kết đầy đủ và dẫn đến hiện tượng quá khớp nghiêm trọng, mô hình áp dụng lớp nén trung bình toàn cục (Global Average Pooling 1D). Lớp này tính toán giá trị trung bình toán học của các khung thời gian, co gọn cấu trúc không gian ma trận về dạng một Vector Đặc trưng duy nhất có kích thước cố định là 1x64 cho mỗi nhánh. Giải pháp thiết kế này giúp triệt tiêu hiện tượng bùng nổ tham số, giải phóng tối đa áp lực bộ nhớ đệm cho bộ vi xử lý của thiết bị biên.

Khối hợp nhất



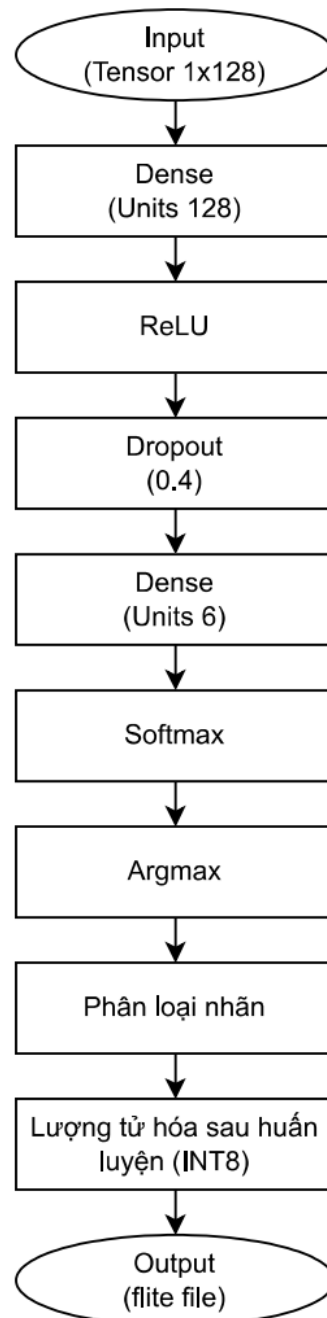
Hình 3.7: Lưu đồ khối hợp nhất chú ý chéo thông minh

Trong thực tế sản xuất tại phân xưởng, dữ liệu âm thanh luôn bị ô nhiễm nặng nề bởi các nguồn tạp âm ngẫu nhiên truyền qua không khí. Nếu sử dụng phương pháp hợp nhất nổi thẳng thông thường, toàn bộ các thuộc tính nhiễu môi trường này sẽ đi thẳng vào bộ phân loại, làm loãng thông tin hữu ích và gây ra hiện tượng sai lệch nghiêm trọng trong kết quả chẩn đoán trạng thái động cơ. Khối chú ý chéo đa phương thức được thiết lập để giải quyết triệt để bài toán này thông qua cơ chế tinh lọc thông tin chủ động.

Giải thuật tận dụng bản chất vật lý của cảm biến gia tốc rung động: do được gắn chặt trực tiếp bằng lực từ tính lên vỏ kim loại của gối đỡ vòng bi, kênh dữ liệu rung động chỉ tiếp nhận các xung động lực học nội tại của cốt máy và hoàn toàn miễn nhiễm với các tạp âm truyền qua không khí. Vector đặc trưng rung động sạch V_{feat} kích thước 1×64 sau khi nén sẽ được dẫn truyền qua một tầng tuyến tính kết hợp hàm kích hoạt Sigmoid. Hàm Sigmoid thực hiện nhiệm vụ nén toàn bộ các giá trị đầu ra thô rơi nghiêm ngặt vào đoạn từ 0 đến 1, thiết lập nên một mảng lưới vector trọng số chú ý gọi là Alpha có kích thước 1×64 .

Hệ thống thực hiện phép nhân toán học thành phần trực tiếp giữa vector đặc trưng âm thanh A_{feat} và mảng trọng số Alpha. Về mặt ý nghĩa vật lý, mảng trọng số này hoạt động như một bộ kiểm duyệt thông tin chéo: nếu microphone thu được một tiếng động lạ có năng lượng cao ngoài môi trường nhưng cảm biến rung động xác nhận vỏ máy hoàn toàn tĩnh lặng tại dải tần đó, giá trị trọng số Alpha tương ứng tại vị trí đó sẽ tiến về số 0. Phép toán nhân toán học này ngay lập tức khóa cổng và xóa sổ thành phần nhiễu ra khỏi miền âm thanh trước khi thực hiện ghép nối. Vector âm thanh sau khi tinh lọc nhiễu chéo và vector rung động nguyên bản sẽ được nối chuỗi lại với nhau, tạo ra một Vector Hợp nhất Đa phương thức duy nhất có kích thước cố định là 1×128 .

Tầng phân loại quyết định đầu ra



Hình 3.8: Lưu đồ tầng phân loại quyết định đầu ra

Khối phân loại tiếp nhận vector hợp nhất có kích thước 1x128 để thực hiện pha ra quyết định chẩn đoán cuối cùng. Đầu tiên, dữ liệu được dẫn truyền vào một tầng liên kết đầy đủ gồm 128 nơ-ron kết hợp hàm kích hoạt ReLU. Tầng này đóng vai trò toán học là tổng hợp, ánh xạ và thiết lập mối tương quan phi tuyến chéo giữa các đặc trưng của hai miền cảm biến, thiết lập không gian ranh giới quyết định chung giữa các nhãn lỗi.

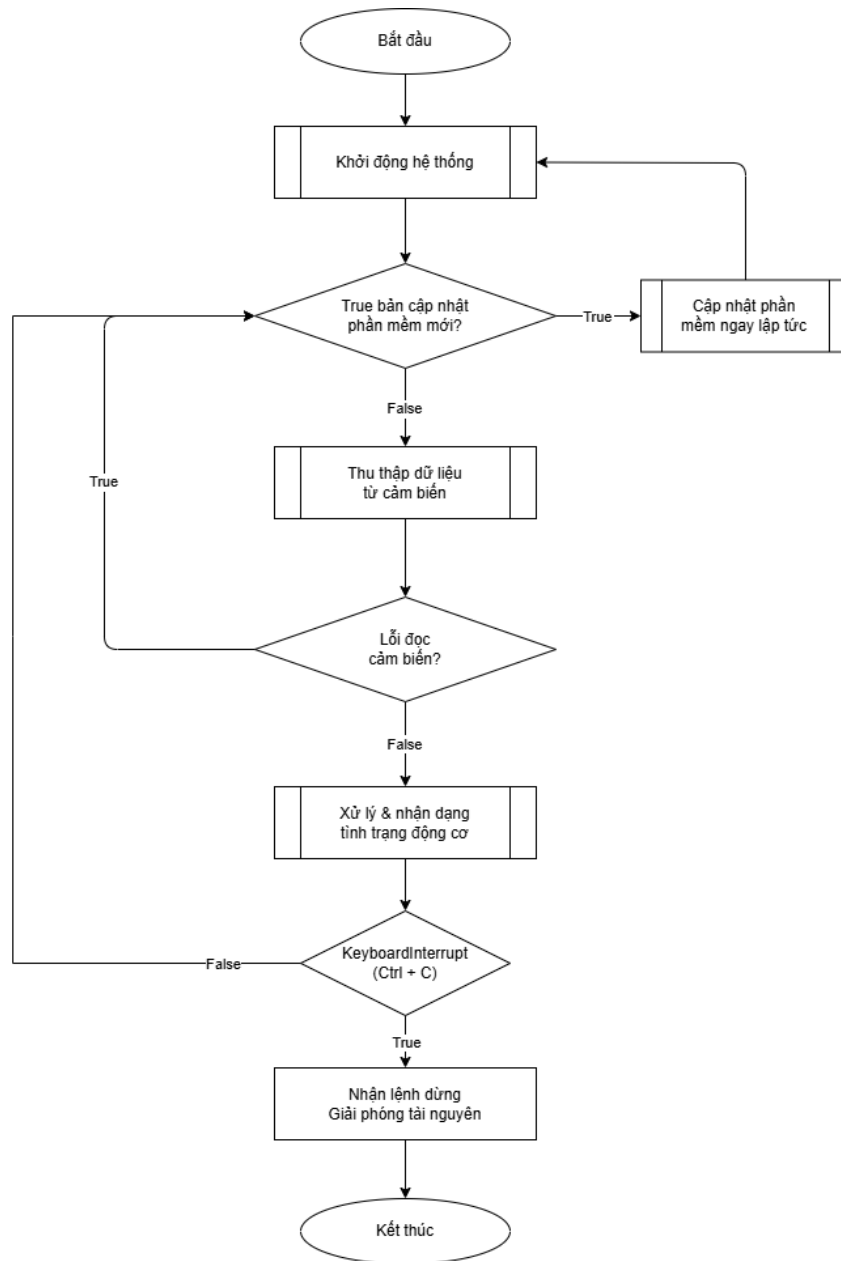
Nhằm gia cố độ bền vững cho mô hình khi triển khai ra thực địa, một lớp ngắt kết nối Dropout thực hiện việc cô lập ngẫu nhiên 40% số lượng nơ-ron trong mỗi chu kỳ huấn luyện, ép buộc các nơ-ron còn lại phải tự học các cấu trúc đặc trưng độc lập, tăng cường khả năng tổng quát hóa dữ liệu và chống lại hiện tượng quá khớp trên phần cứng thực tế.

Tầng quyết định cuối cùng cấu trúc gồm 6 nơ-ron liên kết đầy đủ tương ứng với 6 trạng thái hoạt động của động cơ, tích hợp hàm kích hoạt Softmax. Hàm Softmax thực hiện chuẩn hóa các giá trị đầu ra thành một phân phối xác suất có tổng bằng 1. Nhãn trạng thái sở hữu giá trị xác suất cao nhất sẽ được trích xuất bằng hàm tìm vị trí cực đại để hoàn tất chu trình toán học suy luận của mô hình mạng.

Sau khi hoàn thành pha huấn luyện với bộ tối ưu hóa Adam, toàn bộ đồ thị mạng được cấu hình lượng tử hóa sau huấn luyện (Post-Training Quantization). Quá trình này ép toàn bộ các tham số trọng số của mạng từ dạng số thực FLOAT32 sang định dạng số nguyên INT8, lúc này kích thước mô hình sau khi nén đã giảm đi 4 lần, điều này giúp tối ưu hóa dung lượng bộ nhớ và đẩy tốc độ suy luận đạt trạng thái tối đa.

3.4.3 Lưu đồ giải thuật chính của khối xử lý trung tâm

Khối xử lý trung tâm của hệ thống là Raspberry Pi 4, toàn bộ chương trình được xây dựng bằng ngôn ngữ Python và chạy trên hệ điều hành Raspberry Pi OS. Sau khi khởi động, hệ thống tiến hành nạp các thư viện cần thiết, khởi tạo giao tiếp với cảm biến ICM-42688 thông qua SPI, cảm biến INMP441 thông qua I2S và nạp mô hình học sâu TensorFlow Lite vào bộ nhớ. Sau khi hoàn thành quá trình khởi tạo, chương trình sẽ thực hiện vòng lặp xử lý liên tục cho đến khi hệ thống dừng hoạt động. Lưu đồ giải thuật toàn hệ thống được thể hiện ở Hình 3.9.



Hình 3.9: Lưu đồ giải thuật toàn hệ thống

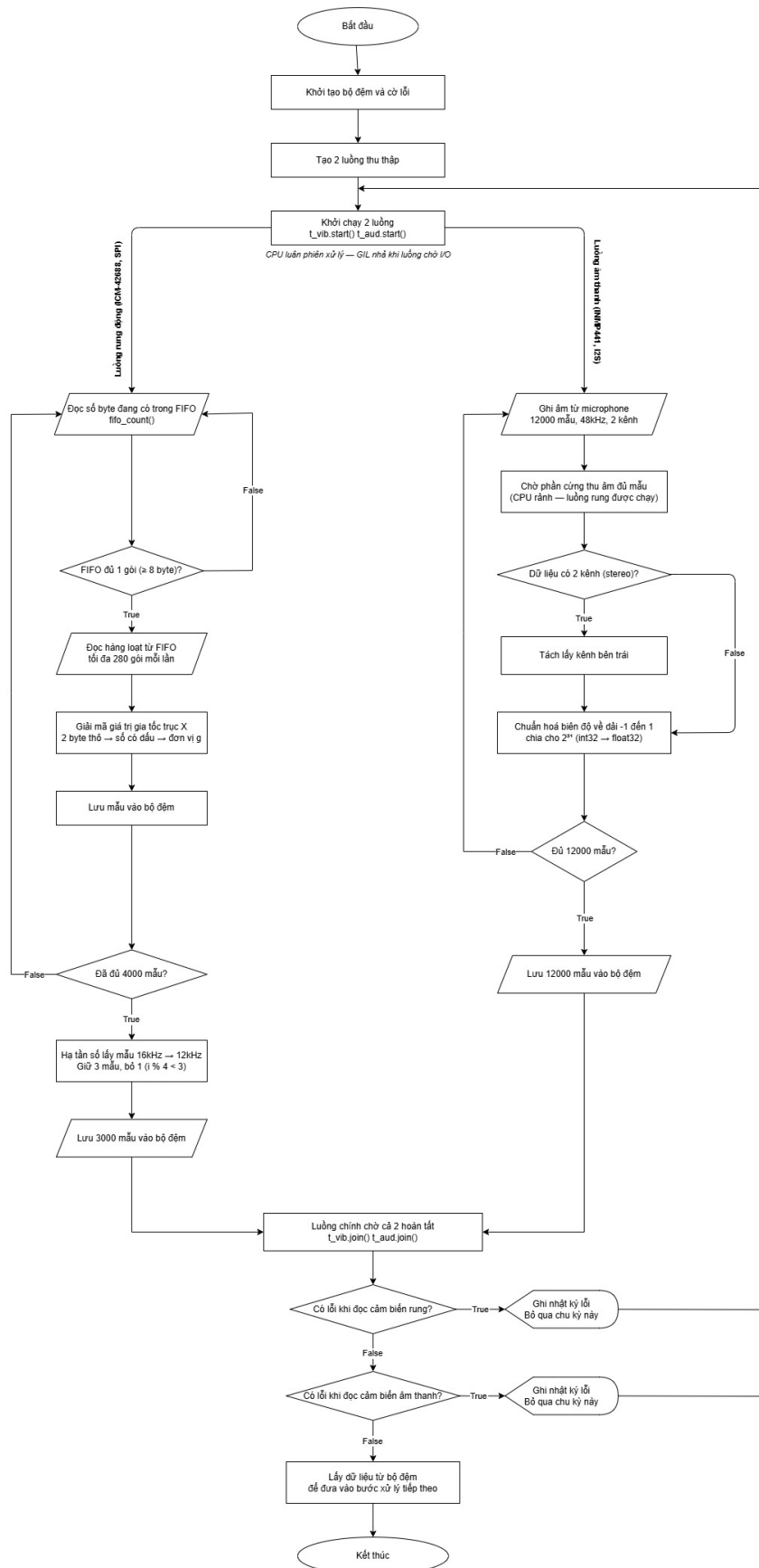
Khi khởi động, hệ thống sẽ tiến hành việc ghi PID bằng hàm `os.getpid()`, `os.getpid()` trả về số định danh tiến trình (PID) của file hệ thống đang chạy, việc này phục vụ cho cơ chế OTA hot-reload, cho phép hệ thống tải lại model mới mà không cần dừng chương trình. Đồng thời, SIGUSR1 dùng để ra hiệu tùy ý giữa các tiến trình, khi hệ thống nhận được tín hiệu SIGUSR1 từ bất kỳ tiến trình nào, nó sẽ gọi hàm để bật cờ thông báo có bản cập nhật thay vì bị dừng đột ngột. Sau đó, hệ thống kiểm tra file mô hình học sâu đã có sẵn trên máy tính hay chưa, nếu đã có thì nạp chương trình vào bộ nhớ bằng hàm `load_model()` tiến hành khởi động cảm biến và hiển thị các thông số của hệ thống như:

tên và tần số hoạt động của cảm biến, đường dẫn file mô hình đang chạy.

Về quá trình thu dữ liệu từ cảm biến, lưu đồ được thể hiện ở Hình 3.10 mục tiêu đặt ra là hệ thống thực hiện đồng thời việc thu thập dữ liệu rung động và dữ liệu âm thanh từ động cơ tại cùng thời điểm. Dữ liệu rung động được lấy từ cảm biến ICM-42688 với tần số lấy mẫu 12 kHz, dữ liệu âm thanh được thu nhận từ cảm biến INMP441 với tần số lấy mẫu 48 kHz. Hệ thống cần thu dữ liệu từ 2 cảm biến trong cùng một khoảng thời gian 250ms để đảm bảo tín hiệu rung động và âm thanh phản ánh cùng một trạng thái của động cơ. Nếu thu tuần tự, đọc rung xong rồi mới đọc âm thanh thì tổng thời gian sẽ là 500ms, và 2 đoạn dữ liệu không còn tương ứng nhau về mặt thời gian. Do đó, để có được dữ liệu đồng thời từ 2 cảm biến, nhóm dùng threading.Thread — nằm trong module threading, là thư viện chuẩn của Python, start() là phương thức của class Thread, khi dùng start() cho 2 cảm biến, tạo ra 2 luồng hoạt động xen kẽ:

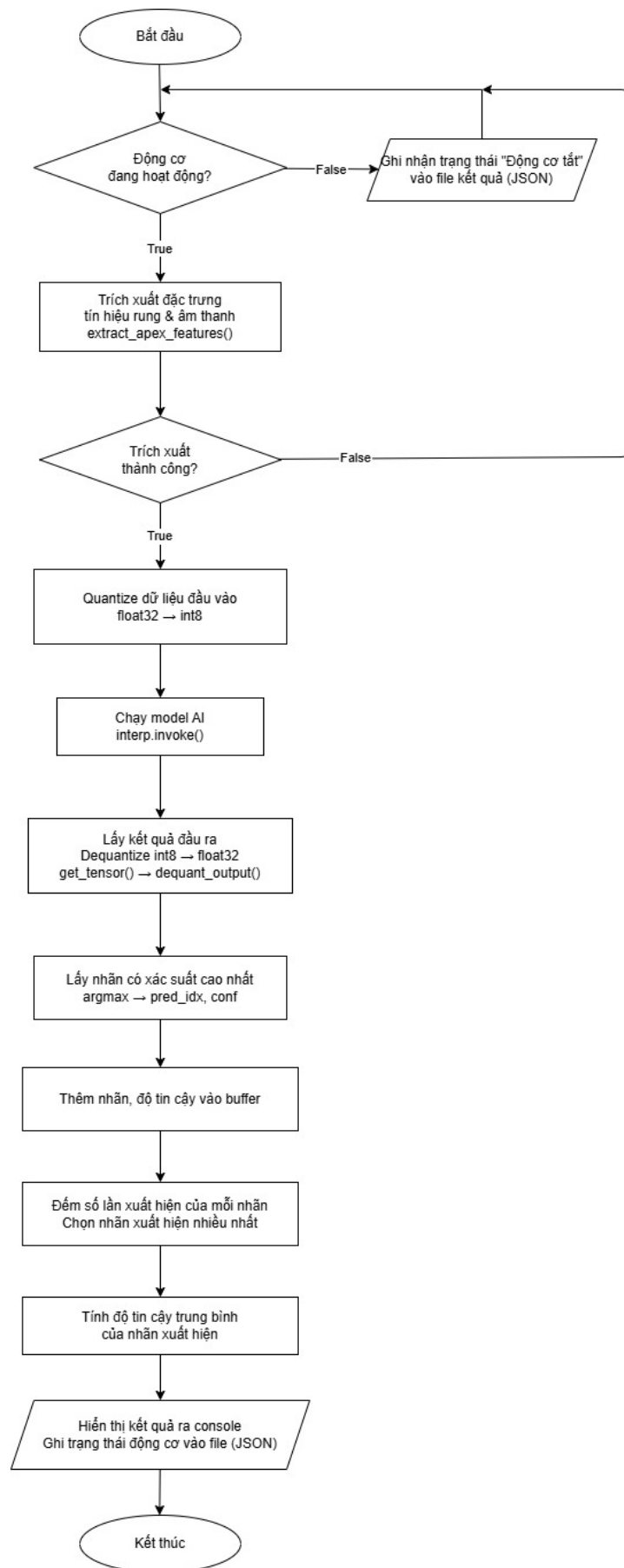
- Luồng rung động: mỗi vòng lặp đọc FIFO, nếu chưa đủ dữ liệu thì gọi time.sleep(0.005). Ngay khi sleep được gọi, hệ điều hành chuyển CPU sang luồng âm thanh.
- Luồng âm thanh: sd.rec() gọi xuống thư viện C của PortAudio. CPU chuyển sang luồng rung động để tiếp tục đọc FIFO.

Kế tiếp, dùng .join() cũng là phương thức có sẵn, khi đó luồng chính chờ đến khi luồng này kết thúc mới tiếp tục chương trình.



Hình 3.10: Lưu đồ giải thuật quá trình thu dữ liệu của cảm biến

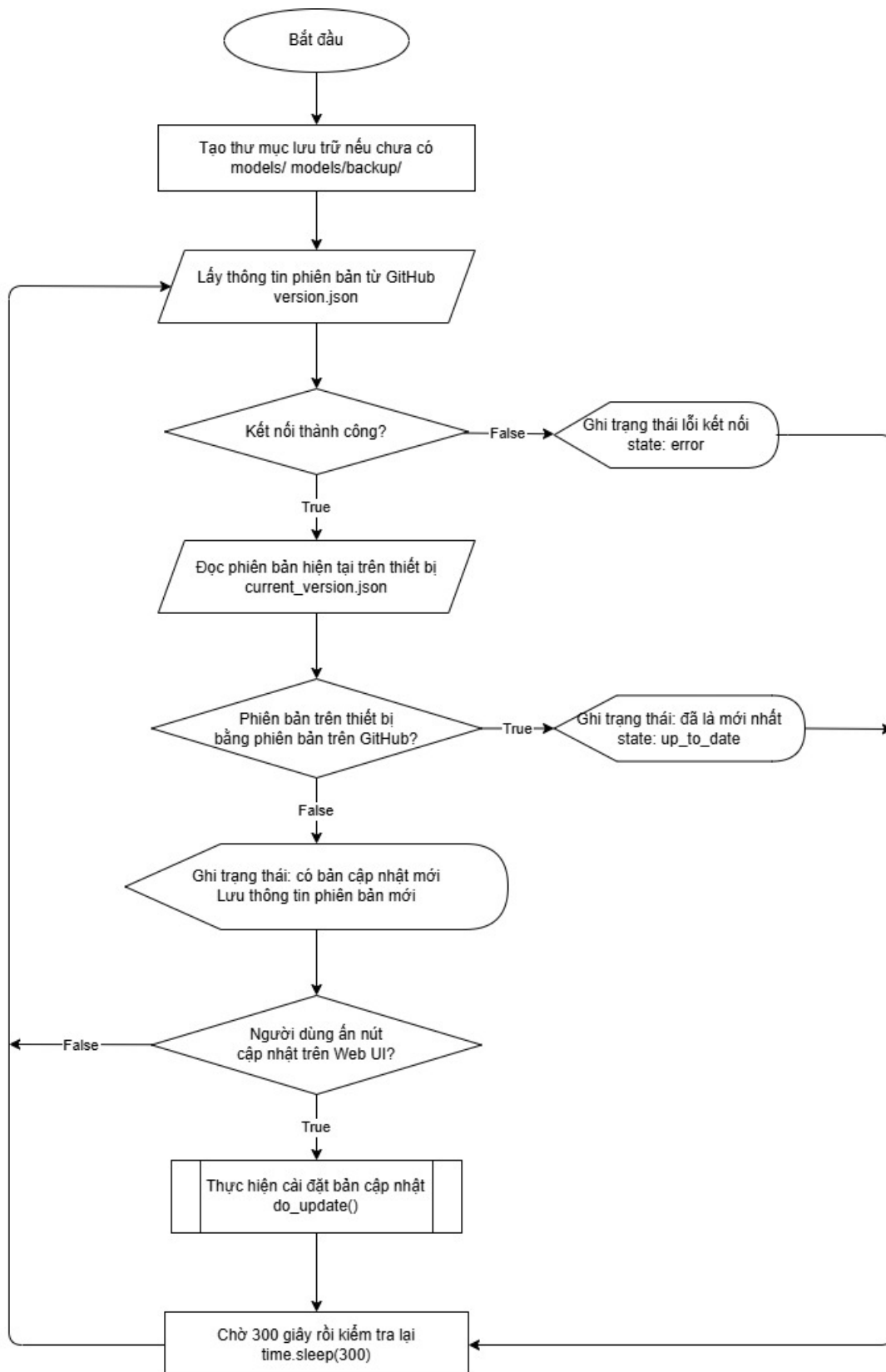
Sau khi thu thập dữ liệu, hệ thống thực hiện kiểm tra trạng thái hoạt động của động cơ thông qua hàm `is_motor_running()` dựa trên độ lệch chuẩn của tín hiệu rung động, có lưu đồ ở Hình 3.11. Nếu giá trị thấp hơn ngưỡng 0.05g, hệ thống kết luận động cơ đang dừng, ghi nhận trạng thái vào file kết quả và bỏ qua chu kỳ hiện tại. Nếu động cơ đang hoạt động, dữ liệu được chuyển sang hàm `extract_apex_features()` để trích xuất đặc trưng PCEN cho cả tín hiệu rung động và âm thanh. Nếu quá trình trích xuất thất bại, chu kỳ hiện tại bị bỏ qua. Các đặc trưng hợp lệ được quantize bởi hàm `prepare_input()` và gán vào tensor đầu vào thông qua `set_tensor()` trước khi gọi `interp.invoke()` để thực hiện suy luận. Kết quả đầu ra được lấy qua `get_tensor()` rồi dequantize bởi `dequant_output()` để lấy nhãn có xác suất cao nhất qua `argmax()`. Để tăng độ ổn định, hệ thống áp dụng cơ chế voting thông qua hàm `vote()` trên cửa sổ 5 chu kỳ gần nhất: nhãn xuất hiện nhiều nhất được chọn làm kết quả cuối cùng, độ tin cậy là trung bình của các lần dự đoán ra nhãn đó. Kết quả được hiển thị ra console và ghi vào file `latest_result.json` để web UI cập nhật theo thời gian thực.



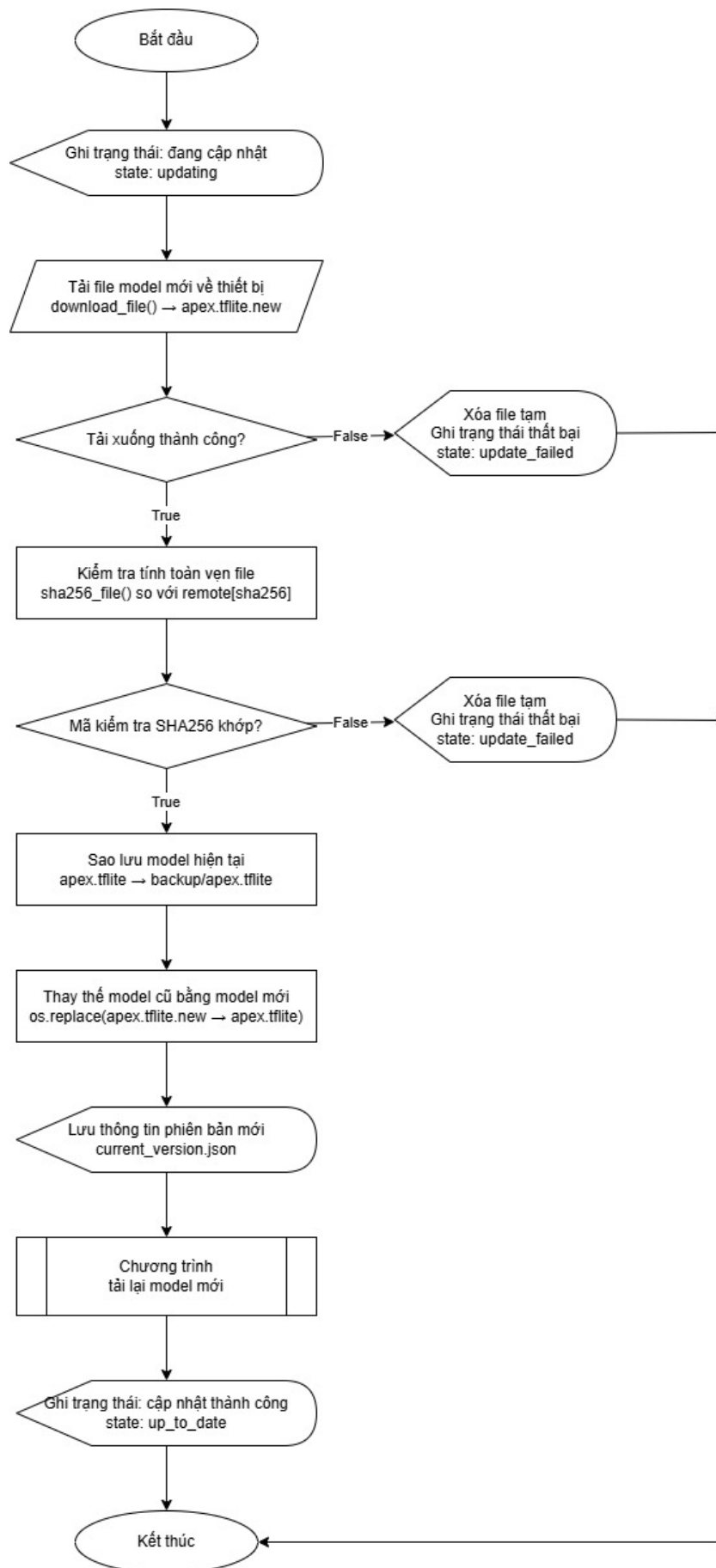
Hình 3.11: Lưu đồ giải thuật quá trình xử lý và nhận dạng tình trạng động cơ

Ngoài chức năng chẩn đoán, hệ thống còn tích hợp cơ chế Over-The-Air (OTA). Chức năng này được thiết kế gồm ba thành phần phối hợp với nhau:

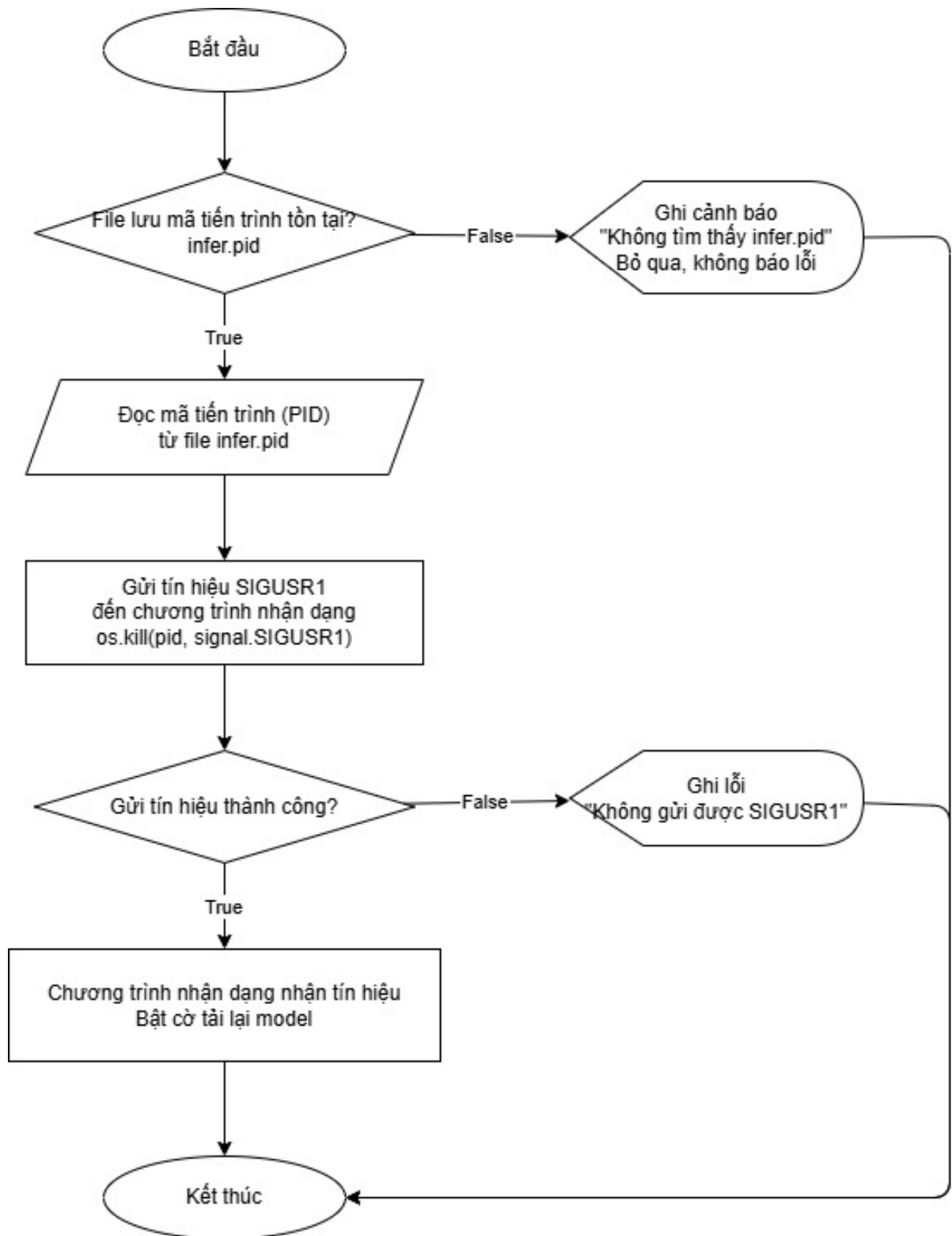
- Thành phần đầu tiên là hàm `check_update()`, có lưu đồ ở Hình 3.12, chạy tự động mỗi 300 giây, kết nối GitHub lấy `version.json` rồi so sánh với phiên bản hiện tại trên thiết bị. Kết quả so sánh được ghi vào `ota_status.json` với trạng thái `up_to_date` hoặc `update_available` kèm changelog. Khi có bản mới, hệ thống chờ người dùng xác nhận trên Web UI trước khi tiến hành cài đặt.
- Thành phần thứ hai là hàm `do_update()`, có lưu đồ ở Hình 3.13, được gọi khi người dùng bấm nút cài đặt. Hàm tải `apex.tflite` mới về thiết bị, kiểm tra tính toàn vẹn bằng SHA256 thông qua `sha256_file()`, sao lưu model cũ vào `backup/` rồi thay thế bằng model mới. Nếu bất kỳ bước nào thất bại, file tạm bị xóa và trạng thái `update_failed` được ghi lại.
- Thành phần thứ ba là hàm `notify_infer_reload()`, có lưu đồ như ở Hình 3.14 gửi tín hiệu `SIGUSR1` đến chương trình nhận dạng thông qua `os.kill()`. Khi nhận được tín hiệu, chương trình nhận dạng bật cờ `_ota_reload_flag` và tải model mới vào bộ nhớ ở chu kỳ tiếp theo mà không cần khởi động lại.



Hình 3.12: Lưu đồ hàm `check_update()`



Hình 3.13: Lưu đồ hàm do_update()



Hình 3.14: Lưu đồ hàm `notify_infer_reload()`

Chương 4

Kết quả

4.1 Kết quả hệ thống

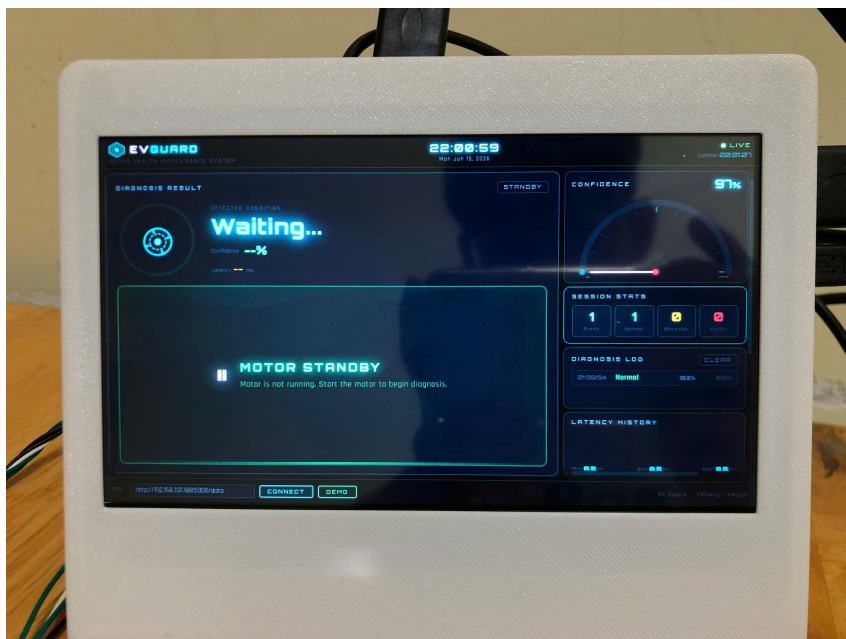
Sau khi hoàn thành việc tích hợp phần cứng và phần mềm, hệ thống chẩn đoán lỗi động cơ xe điện được vận hành thử nghiệm. Kết quả cho thấy hệ thống có khả năng thu thập đồng thời dữ liệu rung động và âm thanh theo thời gian thực, thực hiện phân tích dữ liệu bằng mô hình trí tuệ nhân tạo và hiển thị kết quả trực quan trên giao diện giám sát. Sản phẩm cuối cùng có mặt trước như Hình 4.1 và mặt sau như Hình 4.2.



Hình 4.1: Mặt trước sản phẩm



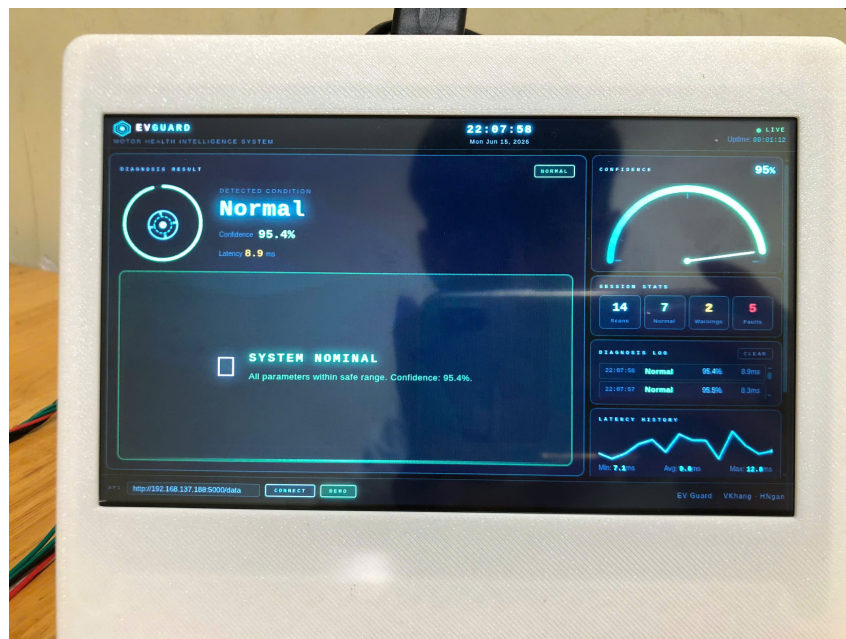
Hình 4.2: Mặt sau sản phẩm



Hình 4.3: Trạng thái standby

Để đưa hệ thống vào trạng thái hoạt động, người dùng cấp nguồn cho thiết bị và khởi động chương trình chẩn đoán trên Raspberry Pi 4, giao diện khi chưa nhận được tín hiệu từ động cơ sẽ hiển thị trạng thái Standby như Hình 4.3. Trong quá trình khởi tạo, hệ thống tự động kiểm tra kết nối với cảm biến rung động ICM-42688-P và cảm biến âm thanh INMP441, đồng thời nạp mô hình trí tuệ nhân tạo vào bộ nhớ. Sau khi hoàn tất

quá trình khởi tạo, hệ thống bắt đầu thu thập dữ liệu từ hai cảm biến và thực hiện suy luận liên tục theo từng đoạn tín hiệu có độ dài khoảng 250 ms. Kết quả dự đoán được cập nhật trực tiếp lên giao diện giám sát theo thời gian thực. Người dùng có thể quan sát trạng thái hoạt động của hệ thống thông qua các thông tin hiển thị trên giao diện mà không cần thao tác cấu hình phức tạp.



Hình 4.4: Hệ thống trả kết quả động cơ ở trạng thái Normal

Hệ thống có khả năng nhận diện sáu trạng thái hoạt động của động cơ gồm:

- Normal: Động cơ hoạt động bình thường.
- HeavyLoad: Động cơ đang hoạt động dưới tải trọng lớn.
- InnerRace: Xuất hiện lỗi tại vòng trong ổ bi.
- OuterRace: Xuất hiện lỗi tại vòng ngoài ổ bi.
- BallFault: Xuất hiện lỗi tại bi lăn.
- Broken: Động cơ hoặc cụm ổ bi có dấu hiệu hư hỏng nghiêm trọng.

Khu vực trung tâm của giao diện hiển thị kết quả chẩn đoán động cơ. Đây là thành phần quan trọng nhất của hệ thống, cho biết trạng thái hoạt động hiện tại dựa trên dữ liệu rung động và âm thanh thu thập được. Chẳng hạn, Hình 4.4 cho thấy trạng thái Normal (hoạt động bình thường) của động cơ và vòng bi.

Bên cạnh kết quả phân loại, hệ thống còn hiển thị giá trị độ tin cậy của dự đoán dưới

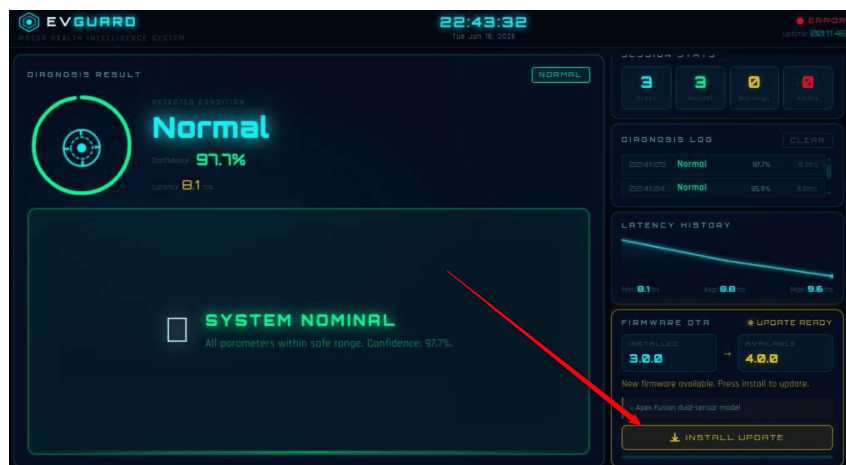
dạng phần trăm. Ngoài ra, giao diện còn cung cấp thông tin về thời gian suy luận của mỗi chu kỳ xử lý. Thông số này cho phép đánh giá khả năng đáp ứng thời gian thực của hệ thống trong quá trình vận hành.



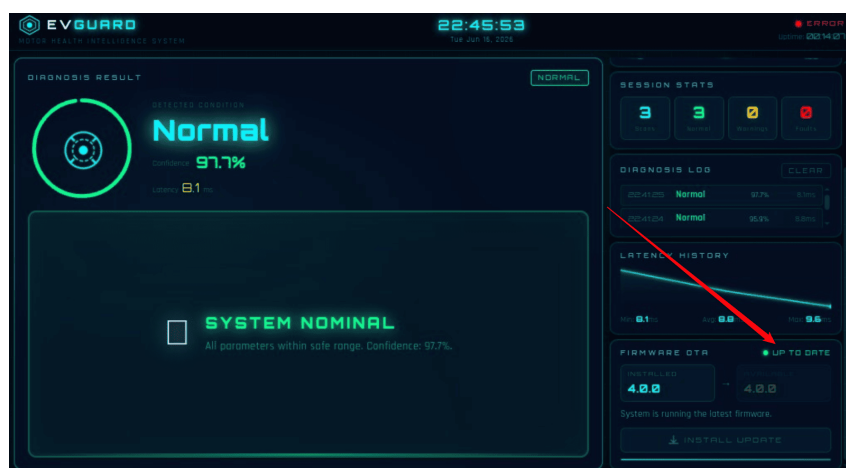
Hình 4.5: Động cơ ở trạng thái Heavyload

Khi động cơ hoạt động bình thường, giao diện hiển thị trạng thái màu xanh lá. Khi động cơ hoạt động ở tải trọng lớn, giao diện chuyển sang màu vàng nhằm nhắc nhở người dùng theo dõi thêm. Đối với các trường hợp phát hiện lỗi ổ bi hoặc hư hỏng nghiêm trọng, giao diện hiển thị màu đỏ để cảnh báo và khuyến nghị kiểm tra thiết bị.

Ngoài ra, hệ thống được tích hợp chức năng cập nhật mô hình từ xa thông qua cơ chế OTA nhằm hỗ trợ nâng cấp mô hình AI sau khi triển khai. Tiến trình OTA định kỳ kiểm tra phiên bản mới từ kho lưu trữ mỗi 5 phút và hiển thị trạng thái cập nhật trên giao diện Dashboard. Khi có phiên bản mới, nút Install Update sẽ sáng lên, người dùng có thể thực hiện cập nhật trực tiếp trên giao diện bằng cách ấn vào nút Install Update (Hình 4.6). Trong quá trình cập nhật, hệ thống vẫn duy trì chức năng chẩn đoán bằng mô hình hiện tại nên không làm gián đoạn việc giám sát. Kết quả thử nghiệm cho thấy chức năng OTA hoạt động ổn định, cho phép cập nhật thành công các phiên bản mô hình mới (Hình 4.7). Đồng thời, cơ chế sao lưu và khôi phục được tích hợp nhằm đảm bảo hệ thống vẫn hoạt động bình thường khi xảy ra lỗi trong quá trình cập nhật.



Hình 4.6: Nút Install Update sáng lên khi có bản cập nhật mới



Hình 4.7: Hệ thống ở trạng thái đã cập nhật

4.2 Kết quả thực hiện phần cứng



Hình 4.8: Bộ xử lý trung tâm được lắp đặt phía sau

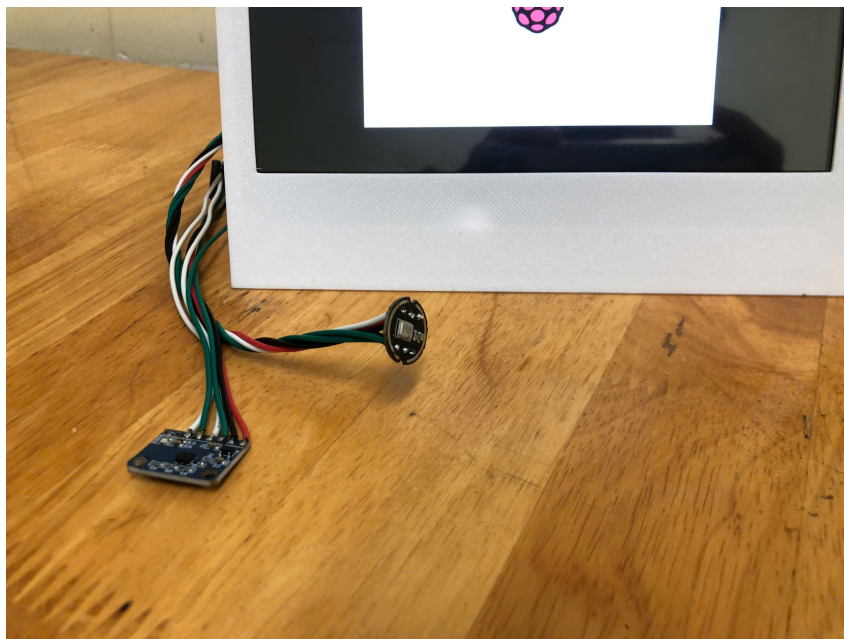
Phần cứng của hệ thống được triển khai trên Raspberry Pi 4 với hai cảm biến chuyên dụng gắn trực tiếp vào động cơ cần giám sát. Bộ xử lý trung tâm là máy tính Raspberry và màn hình giúp giao tiếp với người dùng. Hệ thống được trang bị một màn hình kết nối với Raspberry Pi ở phía sau (Hình 4.8) qua cổng HDMI, đóng vai trò hiển thị trực tiếp giao diện Web Dashboard. Màn hình này cho phép người vận hành quan sát kết quả phân loại tình trạng động cơ theo thời gian thực, độ tin cậy, cảnh báo và lịch sử log. Toàn bộ bo mạch Raspberry Pi 4 cùng màn hình được tích hợp vào một vỏ hộp thi công bằng công nghệ in 3D từ nhựa PLA/ABS. Vỏ hộp được thiết kế riêng để vừa khít với kích thước của Raspberry Pi 4, đồng thời dành chỗ cho các cổng kết nối cảm biến (SPI, I2S), cổng nguồn, cổng HDMI và các lỗ tản nhiệt cần thiết.

Về phần cảm biến, hai cảm biến được nối bằng dây tín hiệu đến bộ xử lý trung tâm như Hình 4.9.

- Cảm biến rung động ICM-42688-P (giao tiếp SPI): Cảm biến được cấu hình ở dải đo $\pm 16g$, tần số lấy mẫu phần cứng 16 kHz, giao tiếp qua bus SPI với tốc độ 8 MHz. Dữ liệu gia tốc trực X được đọc từ bộ đệm FIFO theo từng gói 8 byte, sau đó giảm tần số từ 16 kHz xuống 12 kHz. Mỗi segment thu thập gồm 3.000

mẫu, tương đương 250 ms tín hiệu liên tục. Sau khi khởi động và gọi lệnh `init()`, cảm biến phản hồi giá trị đúng như kỳ vọng, xác nhận kết nối phần cứng thành công.

- Cảm biến âm thanh INMP441 (giao tiếp I2S): Cảm biến microphone MEMS được kết nối qua giao thức I2S tại thiết bị hw:3,0, thu thập ở tần số 48 kHz, 2 kênh stereo. Kênh trái (Left) được lấy làm tín hiệu đầu vào, dữ liệu int32 chuyển sang float32 trước khi xử lý. Số mẫu thu trong mỗi segment tương đương về mặt thời gian với segment rung động (12.000 mẫu/segment).

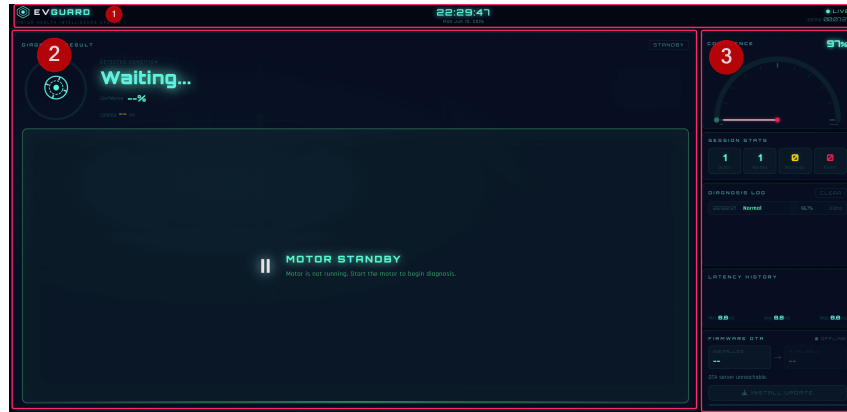


Hình 4.9: Cảm biến được nối với bộ xử lý trung tâm qua các dây tín hiệu

Hai cảm biến được đọc đồng thời bằng cơ chế đa luồng (threading) trong mỗi chu kỳ, đảm bảo dữ liệu rung động và âm thanh phản ánh cùng một khoảng khắc hoạt động của động cơ, không có độ lệch thời gian giữa hai kênh tín hiệu.

Hệ thống còn tích hợp cơ chế phát hiện trạng thái động cơ dựa trên ngưỡng RMS tín hiệu rung động: khi $RMS < 0,05$ g (motor tắt), hệ thống tự động bỏ qua bước suy luận, tránh tiêu tốn tài nguyên tính toán không cần thiết và tránh sinh ra kết quả phân loại vô nghĩa.

4.3 Kết quả nghiên cứu phần mềm



Hình 4.10: Các thành phần trên giao diện

Giao diện của hệ thống được xây dựng bằng HTML, CSS và JavaScript, hoạt động trên nền tảng Flask như Hình 4.10. Dashboard được thiết kế theo phong cách công nghiệp với tông màu tối, giúp người dùng dễ dàng theo dõi trạng thái động cơ một cách trực quan. Bố cục giao diện gồm ba thành phần chính: thanh tiêu đề, khu vực hiển thị thông tin chẩn đoán và thanh cấu hình hệ thống.

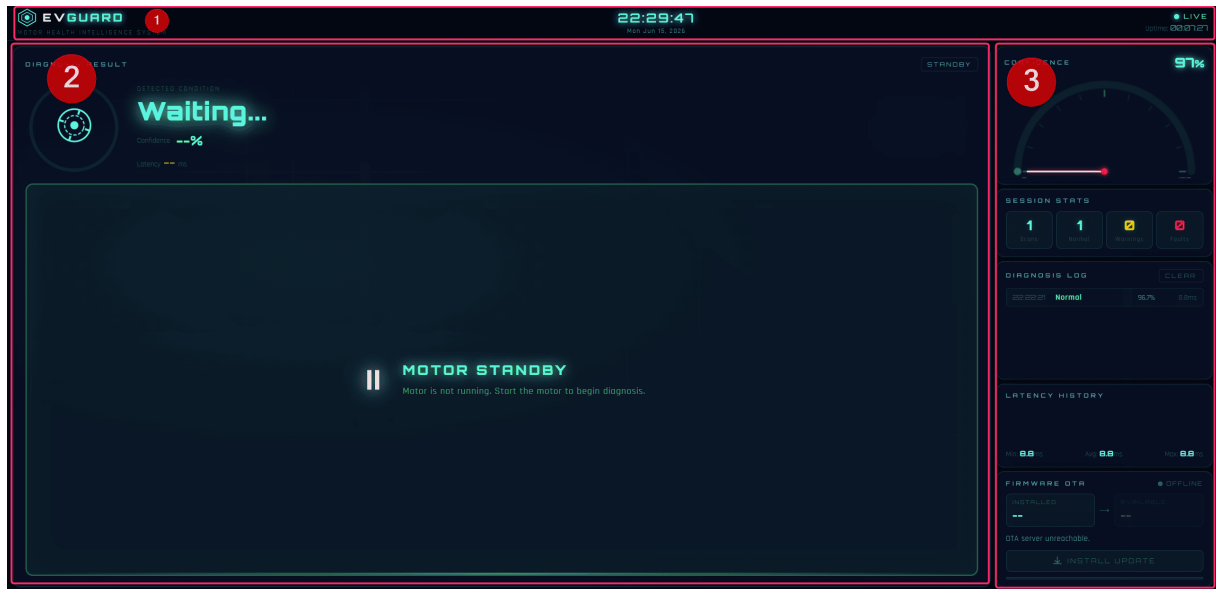
Thanh tiêu đề (vị trí số 1): nằm ở phía trên cùng, hiển thị logo hệ thống, thời gian thực và trạng thái kết nối giữa giao diện với dịch vụ xử lý dữ liệu. Khu vực này giúp người dùng nhanh chóng nhận biết tình trạng hoạt động của hệ thống trong quá trình giám sát.

Khu vực Dashboard chính: được chia thành hai cột. Cột bên trái (vị trí số 2) hiển thị kết quả chẩn đoán theo thời gian thực, bao gồm trạng thái động cơ, độ tin cậy của mô hình, thời gian suy luận và bảng cảnh báo. Màu sắc của các thành phần được thay đổi tương ứng với từng mức độ nghiêm trọng nhằm hỗ trợ người dùng nhận biết nhanh các bất thường phát sinh.

Cột bên phải (vị trí số 3) là khu vực thống kê và theo dõi, bao gồm đồng hồ đo độ tin cậy, các thông số thống kê của phiên làm việc, nhật ký chẩn đoán, biểu đồ lịch sử độ trễ suy luận và bảng quản lý cập nhật mô hình từ xa (OTA). Các thông tin này hỗ trợ người dùng đánh giá hiệu năng hoạt động của hệ thống cũng như theo dõi diễn biến trạng thái động cơ theo thời gian.

Kết quả thử nghiệm cho thấy giao diện hoạt động ổn định trên trình duyệt Chromium

của Raspberry Pi 4 cũng như các thiết bị khác trong cùng mạng nội bộ. Dữ liệu được cập nhật liên tục theo thời gian thực, đáp ứng yêu cầu giám sát và chẩn đoán lỗi động cơ xe điện của đề tài.

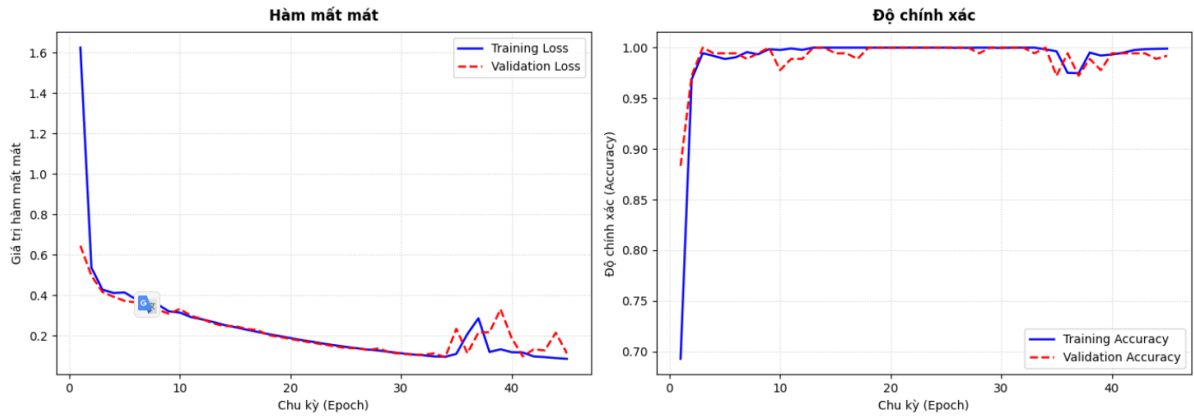


Hình 4.11: Giao diện người dùng

4.4 Thảo luận

4.4.1 Đánh giá kết quả huấn luyện

Quá trình tối ưu hóa và huấn luyện thực nghiệm hệ thống được cấu hình triển khai trong 50 chu kỳ liên tục với kiến trúc mạng tích chập một chiều song song đa phương thức (Apex Fusion 1D-CNN). Các thông số động lực học về độ chính xác và giá trị hàm mất mát trên tập huấn luyện cũng như tập kiểm định được hệ thống tự động ghi nhận chi tiết, làm cơ sở khoa học để đánh giá năng lực trích xuất thuộc tính hình học dải tần của cấu trúc mạng đề xuất.



Hình 4.12: Biểu đồ tiến trình hội tụ

Dựa trên chuỗi số liệu theo dõi thực tế từ tiến trình tối ưu hóa, đồ thị hội tụ trọng số của mô hình mạng lai 1D-CNN được phân tách thành hai giai đoạn chính:

- Giai đoạn khởi động và tăng trưởng gia tốc học (Chu kỳ 1 đến chu kỳ 14): Tại chu kỳ đầu tiên, khi các trọng số của mảng lọc tích chập chưa được định hình, độ chính xác tập huấn luyện đạt 69.27% và tập kiểm định đạt 88.33%. Mô hình hội tụ nhanh cho đến chu kỳ 14, độ chính xác toàn cục trên cả hai tập dữ liệu huấn luyện và kiểm định đã nhanh chóng bộc phát vọt lên ngưỡng tuyệt đối 100%, đồng thời giá trị hàm mất mát sụt giảm sâu từ 1.6238 rơi xuống đáy ổn định ở mức 0.2468. Hiện tượng này khẳng định các tầng lọc tích chập một chiều đã bóc tách vô cùng hiệu quả các dải năng lượng đặc trưng lỗi cốt lõi ngay từ pha khởi động hệ thống.
- Giai đoạn biến động cục bộ và kích hoạt Early Stopping (Chu kỳ 15 đến chu kỳ 45): Do đặc thù dữ liệu đa phương thức thô luôn tồn tại nhiễu nền công nghiệp, đồ thị hàm mất mát xuất hiện các điểm dao động xung biên độ tại chu kỳ 35 và 37, kéo theo sự sụt giảm độ chính xác kiểm định tạm thời xuống ngưỡng 97.22%. Cuối cùng, thuật toán Early Stopping chính thức ngắt tiến trình huấn luyện tại chu kỳ thứ 45 khi nhận diện các trọng số đã đạt trạng thái bão hòa tối ưu để tránh tình trạng quá khớp (Overfitting).

4.4.2 Đánh giá chi tiết hiệu năng suy luận thực nghiệm

Để làm nổi bật giá trị thực tiễn và tính đúng đắn khi lựa chọn mạng tích chập một chiều 1D-CNN đề xuất trong điều kiện thiết bị Edge AI hạn chế tài nguyên, đồ án thực

hiện một chiến dịch thực nghiệm so sánh chéo với 4 cấu trúc thuật toán đối chứng bao gồm: Mạng đồ thị nơ-ron (GNN), Mạng tích chập một chiều (1D-CNN), Mạng tích chập hai chiều (2D-CNN), và Random Forest.

Model	Accuracy
GNN	0.93
1D-CNN Baseline	0.99
2D-CNN	1.00
Random Forest	0.99
Apex Fusion 1D-CNN	0.98

Bảng 4.1: Tổng hợp độ chính xác trên từng loại mô hình

Nhận xét về sự chênh lệch chỉ số giữa mô hình đề xuất và các mô hình baseline: Một điểm đáng chú ý trong kết quả thực nghiệm là mô hình 1D-CNN thô và 2D-CNN đạt độ chính xác lên đến 99% và 100% trên tập dữ liệu sạch của phòng thí nghiệm. Tuy nhiên, mô hình đề xuất Apex Fusion 1D-CNN sau lượng tử hóa vẫn đạt mức độ chính xác tiệm cận tuyệt đối là 98%, phản ánh một năng lực học máy vượt trội trong bài toán học sâu đa phương thức thực tế:

- Các mô hình Baseline đơn lập chỉ hoạt động trên một miền cảm biến rung động cố định trong điều kiện môi trường phòng thí nghiệm tĩnh, lý tưởng. Do đó, mạng nơ-ron rất dễ bị bão hòa và tối ưu tuyệt đối trên tập kiểm thử nội bộ nhưng có thể sụt giảm độ chính xác nghiêm trọng khi đưa ra phân xử thực tế có tạp âm nền kéo dài.
- Ngược lại, mô hình đề xuất Apex Fusion 1D-CNN bắt buộc phải đồng thời xử lý và dung hợp âm thanh truyền qua không khí chứa nhiều nhiễu môi trường và rung động truyền qua vỏ máy. Việc đạt độ chính xác ổn định lên đến 98% trên cả 6 nhãn trạng thái chính là minh chứng cho thấy mô hình đã thực sự học được các đặc trưng động lực học liên kết đa cảm biến, mang lại khả năng kháng nhiễu và độ bền vững vượt trội khi triển khai thực tế.

Bảng 4.2: Tổng hợp so sánh hiệu năng toán học và tài nguyên phần cứng Edge AI

Chỉ số	GNN	1D-CNN	2D-CNN	Random Forest
Dung lượng model	6251.75 KB	1047.37 KB	23345.45 KB	775.01 KB
Số lượng tham số	529,414	84,806	1,986,694	Không áp dụng
Thời gian tiền xử lý	2.18 ms/mẫu	0.05 ms/mẫu	2.48 ms/mẫu	6.44 ms/mẫu
Thời gian suy luận	81.37 ms/mẫu	81.39 ms/mẫu	83.23 ms/mẫu	33.87 ms/mẫu
Tổng độ trễ hệ thống	83.56 ms/mẫu	81.44 ms/mẫu	85.72 ms/mẫu	40.31 ms/mẫu
Thời gian huấn luyện	15.14 s	19.03 s	22.54 s	0.41 s

Từ các kết quả đo lường thực nghiệm trực diện này, đề án chính thức lựa chọn và chốt kiến trúc mạng tích chập một chiều song song (Apex Fusion 1D-CNN) làm giải pháp tối ưu cuối cùng để triển khai cài đặt vào hệ thống chẩn đoán thực tế.

Xét về dung lượng lưu trữ và cấu trúc tham số nơ-ron, mô hình đề xuất 1D-CNN thể hiện ưu thế vượt trội về khả năng tiết kiệm bộ nhớ với dung lượng cực kỳ gọn nhẹ chỉ đạt 1047.37 KB và số lượng tham số tối giản ở mức 84,806. Khi đối chiếu với kiến trúc mạng đồ thị (GNN Baseline) sở hữu dung lượng 6251.75 KB (529,414 tham số) hoặc mạng tích chập hai chiều (2D-CNN Baseline) mang dung lượng công kênh lên tới 23345.45 KB (gần 2 triệu tham số), mô hình đề xuất 1D-CNN giúp giải phóng tối đa áp lực không gian lưu trữ phần cứng. Sự tinh gọn này là điều kiện tiên quyết giúp hệ thống triệt tiêu hoàn toàn các rủi ro về tràn bộ nhớ đệm (RAM) hoặc gây hiện tượng quá nhiệt vi xử lý khi vận hành tuần hoàn lâu dài trên các bo mạch máy tính nhúng hạn chế tài nguyên như Raspberry Pi 4.

Khi đánh giá sâu về vận tốc vận hành và độ trễ hệ thống, mô hình 1D-CNN đề xuất chứng minh năng lực đáp ứng thời gian thực (real-time) công nghiệp một cách lý tưởng. Tổng độ trễ toàn cục của mô hình chỉ tiêu tốn tổng cộng 81.44 ms để hoàn thành một chu kỳ chẩn đoán mẫu dữ liệu đa phương thức. Mốc thời gian này nằm nghiêm ngặt dưới ngưỡng đáp ứng tiêu chuẩn kỹ thuật an toàn (dưới 100 ms), cho phép phần mềm phát hiện và phát lệnh cảnh báo các sự cố va đập cơ khí bộc phát tức thời tại phân xưởng.

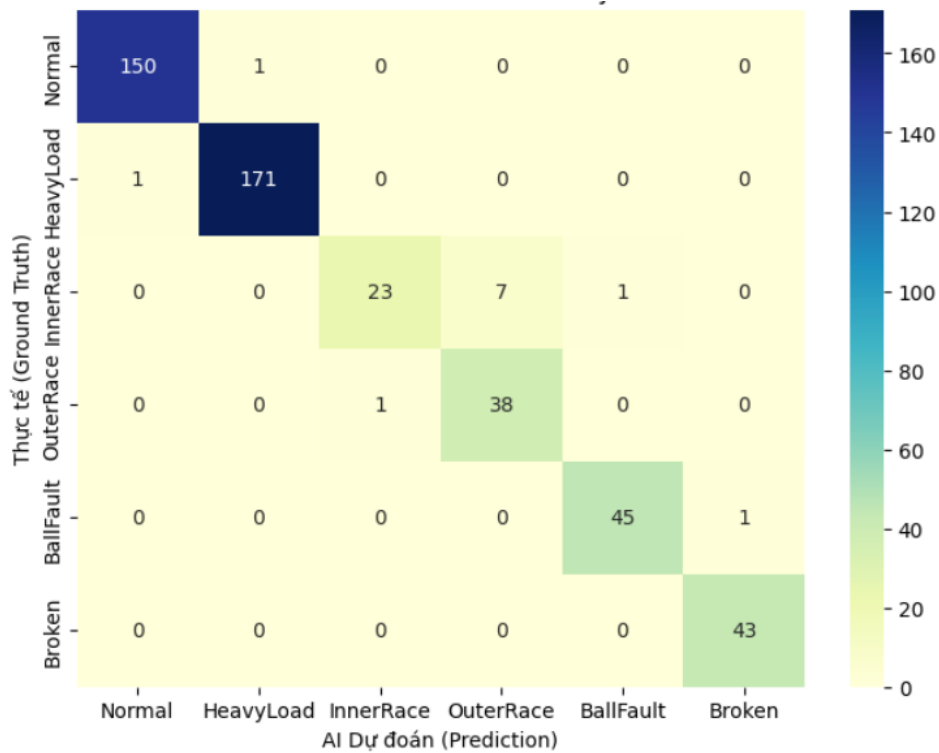
Mặc dù thuật toán học máy truyền thống Random Forest sở hữu tổng độ trễ hệ thống thấp nhất (40.31 ms/mẫu), nhưng kiến trúc mạng học sâu 1D-CNN đề xuất lại vượt trội

hoàn toàn về khả năng kháng nhiễu và độ bền vững ngoài thực địa. Việc áp dụng kỹ thuật tích chập giãn (Dilated Convolution) quét dọc theo trục thời gian giúp mạng nơ-ron có khả năng bao quát ngữ cảnh xung lực chu kỳ tốt hơn các phép tích chập phẳng thông thường. Khi đưa ra môi trường nhà máy tồn tại nhiều tiếng ồn và rung chấn ngẫu nhiên kéo dài, cấu trúc bộ lọc thông minh của các tầng xử lý 1D-CNN kết hợp với cơ chế chú ý chéo (Cross-Attention) sẽ tự động tinh lọc thông tin, dập tắt các dải tạp âm ngẫu nhiên truyền qua không khí để duy trì độ tin cậy chẩn đoán cao ổn định ở mức 98%; tránh được hiện tượng sai lệch dải tần và sụt giảm hiệu năng nghiêm trọng mà các mô hình Baseline thông thường rất dễ gặp phải khi rời khỏi điều kiện phòng thí nghiệm.

	precision	recall	f1-score	support
Normal	0.99	0.99	0.99	151
HeavyLoad	0.99	0.99	0.99	172
InnerRace	0.96	0.74	0.84	31
OuterRace	0.84	0.97	0.90	39
BallFault	0.98	0.98	0.98	46
Broken	0.98	1.00	0.99	43
accuracy			0.98	482
macro avg	0.96	0.95	0.95	482
weighted avg	0.98	0.98	0.97	482

Hình 4.13: Bảng báo cáo đánh giá hiệu năng suy luận chi tiết trên mô hình đề xuất

Để chứng minh năng lực chẩn đoán độc lập khi chạy suy luận thực tế, mô hình Apex Fusion 1D-CNN được đưa vào kiểm chứng nghiêm ngặt trên tập kiểm thử. Kết quả thực nghiệm toàn cục ghi nhận chỉ số chính xác đạt mức 98%, chỉ số F1-Score đạt 0.97, minh chứng cho sự ổn định của mô hình.



Hình 4.14: Ma trận nhầm lẫn trên 300 mẫu kiểm thử

Dựa trên biểu đồ ma trận nhầm lẫn thu được từ thực nghiệm thực tế, năng lực phân loại của mạng 1D-CNN đề xuất thể hiện sự phân hóa rõ ràng:

- Các nhóm trạng thái đạt độ chính xác tuyệt đối: Ba trạng thái lỗi bao gồm Normal, HeavyLoad, Broken đạt tỷ lệ phân loại đúng tuyệt đối 100.00%. Nhóm BallFault cũng đạt tỷ lệ chính xác tiệm cận tối đa với 98.00%. Kết quả này chứng minh các cửa sổ lọc một chiều đã thiết lập được ranh giới phân lớp phẳng cực kỳ vững chắc cho các trạng thái này.
- Quy luật nhầm lẫn động học giữa Lỗi vòng trong và Lỗi vòng ngoài: Ma trận chẩn đoán chỉ ra một điểm nghẽn mang tính quy luật động lực học gồ ghề giữa hai nhãn InnerRace và OuterRace. Cụ thể, nhãn InnerRace có 3 mẫu bị đoán sai sang OuterRace (và 1 mẫu sang BallFault).

Hiện tượng nhầm lẫn trên hoàn toàn tuân theo bản chất vật lý lan truyền sóng cơ học chứ không mang tính ngẫu nhiên lỗi thuật toán. Khi vòng trong của vòng bi xoay quanh trục ở tốc độ cao, các vết nứt bề mặt sẽ liên tục va đập vào hệ thống con lăn, phát sinh các chuỗi xung lực. Các xung động này bắt buộc phải truyền dẫn xuyên qua màng dầu

bôi trơn, qua hệ thống con lăn rồi mới tác động đến vòng ngoài cố định của gối đỡ và truyền tới vỏ máy nơi đặt cảm biến gia tốc. Trong một số khung thời gian làm việc đặc thù dưới tải trọng dao động, dạng phổ năng lượng mật độ đỉnh xung và dải tần số cộng hưởng của dòng xung lực lan truyền từ vòng trong bị suy hao, biến dạng thành cấu trúc tương đồng cấu trúc phổ lỗi trực tiếp tại vòng ngoài. Tuy nhiên, nhờ sự gán vác phối hợp thông tin từ kênh âm thanh qua cơ chế chú ý, mô hình vẫn kiểm soát rủi ro, khóa chặt các sai lệch không để nhầm lẫn sang các nhãn an toàn như Normal hay HeavyLoad, đảm bảo hệ thống cảnh báo vận hành an toàn tuyệt đối tại biên.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Sau quá trình tìm tòi, nghiên cứu thực hiện đề tài "THIẾT KẾ VÀ THI CÔNG HỆ THỐNG CHẨN ĐOÁN LỖI CHO ĐỘNG CƠ XE ĐIỆN", nhóm đã xây dựng được một hệ thống đạt được những yêu cầu đưa ra như sau:

- Về chức năng kỹ thuật, hệ thống có khả năng thu thập dữ liệu cảm biến, từ đó thực hiện phân loại và chẩn đoán lỗi động cơ theo thời gian thực. Kết quả chẩn đoán cùng trạng thái hệ thống được hiển thị trực quan trên giao diện người dùng, đồng thời hệ thống hỗ trợ cập nhật phần mềm từ xa thông qua OTA Server, tạo điều kiện thuận lợi cho việc bảo trì và nâng cấp sau triển khai.
- Về thông số kỹ thuật, hệ thống hoạt động với nguồn cấp 5V DC, có kích thước nhỏ gọn xấp xỉ $20 \times 15 \times 15$ cm, phù hợp để tích hợp vào các thiết bị thực tế. Thời gian khởi động sau khi cấp nguồn khoảng 10–20 giây và thời gian xử lý, hiển thị kết quả chẩn đoán đáp ứng yêu cầu thời gian thực. Tính năng cập nhật OTA qua kết nối WiFi giúp hệ thống linh hoạt trong việc cải tiến và mở rộng chức năng mà không cần can thiệp trực tiếp vào phần cứng.

Tuy nhiên, nhóm cũng nhận thấy một số hạn chế của hệ thống như khả năng mở rộng cần được kiểm chứng thêm trong các môi trường thực tế với quy mô lớn hơn và các tình huống vận hành phức tạp hơn.

Các hướng phát triển tiếp theo cho đề tài bao gồm:

- Hệ thống có thể được mở rộng số lượng và chủng loại cảm biến để thu thập thêm nhiều thông tin từ động cơ như nhiệt độ, dòng điện, điện áp hoặc tốc độ quay.
- Mô hình học sâu hiện tại có thể tiếp tục được cải tiến bằng cách sử dụng các kiến trúc hiện đại hơn. Việc tối ưu mô hình sẽ giúp nâng cao khả năng nhận dạng lỗi phức tạp và cải thiện hiệu suất xử lý thời gian thực.
- Hệ thống hiện được triển khai trên Raspberry Pi 4, trong tương lai có thể nghiên cứu tích hợp các bộ tăng tốc AI như TPU hoặc GPU nhúng để tăng tốc quá trình suy luận mô hình học sâu. Điều này giúp giảm thời gian xử lý và nâng cao khả năng triển khai trên các ứng dụng công nghiệp.
- Có thể phát triển hệ thống theo hướng điện toán đám mây kết hợp IoT, cho phép lưu trữ dữ liệu tập trung và giám sát từ xa thông qua Web App hoặc ứng dụng di động. Dữ liệu thu thập từ nhiều thiết bị có thể được tổng hợp để xây dựng cơ sở dữ liệu lớn phục vụ huấn luyện và tối ưu thuật toán.
- Hệ thống OTA hiện sử dụng GitHub làm máy chủ lưu trữ cập nhật. Trong tương lai có thể phát triển máy chủ OTA chuyên dụng nhằm tăng tính bảo mật, quản lý phiên bản tốt hơn và hỗ trợ cập nhật đồng thời nhiều thiết bị trong hệ thống.
- Hệ thống có thể được triển khai và kiểm thử trên nhiều loại động cơ xe điện khác nhau trong môi trường thực tế nhằm đánh giá độ ổn định, khả năng mở rộng và tính ứng dụng của hệ thống trong các bài toán chẩn đoán công nghiệp.

Nhìn chung, đề tài đã đạt được các mục tiêu chính đặt ra, đồng thời cung cấp một giải pháp thiết thực cho bài toán Chẩn đoán lỗi cho động cơ xe điện.

Tài liệu tham khảo

- [1] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Wiley, 2011.
- [2] A. K. S. Jardine, D. Lin, and D. Banjevic, “A review on machinery diagnostics and prognostics implementing condition-based maintenance,” *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.
- [3] D. T. Hoang and H. J. Kang, “A survey on deep learning based bearing fault diagnosis,” *Neurocomputing*, vol. 335, pp. 327–335, 2019.
- [4] Y. Lei, J. Lin, Z. He, and M. J. Zuo, “A review on empirical mode decomposition in fault diagnosis of rotating machinery,” *Mechanical Systems and Signal Processing*, vol. 35, no. 1–2, pp. 108–126, 2013.
- [5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [6] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [7] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.
- [8] S. W. Smith, *The Scientist and Engineer’s Guide to Digital Signal Processing*. California Technical Publishing, 1997.
- [9] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

- [10] B. Logan, “Mel Frequency Cepstral Coefficients for music modeling,” in *Proceedings of the International Symposium on Music Information Retrieval (ISMIR)*, 2000, pp. 1–11.
- [11] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., 2024, draft edition.